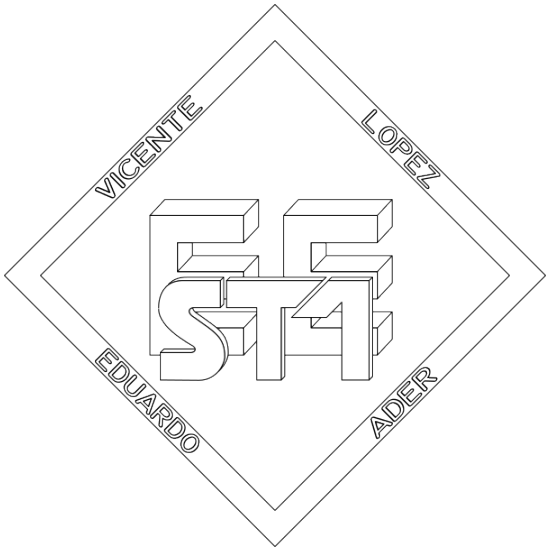


Escuela de Educación Secundaria Técnica N° 1
de Vicente López — Eduardo Ader

Aplicaciones de Electrónica Digital II y III — 5° Año

Apuntes de Cátedra

Docente: Ariel Wels

Ciclo Lectivo 2025

Índice general

1. Lógica Secuencial **6**

1.1. Lógica combinacional vs. lógica secuencial . . . 6
1.2. Latches . . . 6
1.2.1. Latch SR (Set-Reset) . . . 6
1.2.2. Latch SR con habilitación . . . 9
1.2.3. Latch D con habilitación . . . 10
1.3. Flip-flops . . . 11
1.3.1. Flip-flop SR disparado por flanco . . . 11
1.3.2. Flip-flop D disparado por flanco . . . 12
1.3.3. Flip-flop JK disparado por flanco . . . 12
1.3.4. Flip-flop T (Toggle) . . . 13
1.3.5. Entradas asíncronas: Preset y Clear . . . 14
1.3.6. Características de funcionamiento . . . 14
1.4. Registros de desplazamiento . . . 14
1.4.1. Entrada serie / Salida serie (SISO) . . . 15
1.4.2. Entrada serie / Salida paralelo (SIPO) . . . 16
1.4.3. Entrada paralelo / Salida serie (PISO) . . . 16
1.4.4. Entrada paralelo / Salida paralelo (PIPO) . . . 17
1.4.5. Registros bidireccionales . . . 17
1.4.6. Contadores basados en registros de desplazamiento . . . 17
1.5. Contadores . . . 18
1.5.1. Contadores asíncronos (con propagación) . . . 18
1.5.2. Contadores síncronos . . . 19
1.5.3. Contadores ascendentes/descendentes . . . 19
1.5.4. Módulo de un contador y truncamiento de secuencia . . . 19
1.5.5. Contadores en cascada . . . 19
1.5.6. Decodificación de estados . . . 20
1.6. Máquinas de estado . . . 20
1.6.1. Diagrama de estados . . . 20
1.6.2. Máquina de Moore . . . 21
1.6.3. Máquina de Mealy . . . 21
1.6.4. Tabla de estados . . . 21
1.6.5. Proceso de diseño de una máquina de estados . . . 21
1.7. Actividades . . . 22
1.7.1. Latches . . . 22
1.7.2. Flip-flops . . . 23
1.7.3. Registros de desplazamiento . . . 24
1.7.4. Contadores . . . 25

1.7.5. Guía de resolución 26

2. Memorias Digitales **31**
2.1. ¿Qué es una memoria? 31
2.2. Unidades de datos binarios 31
2.3. Organización interna: la matriz de memoria 32
2.4. Direccionamiento 33
2.5. Operaciones básicas: lectura y escritura 33
2.5.1. Operación de escritura 34
2.5.2. Operación de lectura 34
2.6. Clasificación de las memorias 35
2.7. Memorias RAM 36
2.7.1. RAM estática (SRAM) 36
2.7.1.1. La idea: una celda hecha con un latch D 36
2.7.1.2. La optimización: celda de 6 transistores (6T) 37
2.7.2. RAM dinámica (DRAM) 38
2.8. Memorias ROM y sus variantes 39
2.8.1. ROM de máscara 39
2.8.2. PROM (*Programmable ROM*) 39
2.8.3. EPROM (*Erasable PROM*) 39
2.8.4. EEPROM (*Electrically Erasable PROM*) 40
2.8.5. Memoria Flash 40
2.9. Expansión de memoria 41
2.9.1. Expansión de palabra (ampliación de bits por posición) 41
2.9.2. Expansión de capacidad (ampliación de direcciones) 42
2.9.3. Expansión combinada 42

3. Sistemas Microprogramables **43**
3.1. Lógica cableada y lógica programada 43
3.2. Definición de sistema microprogramable 43
3.3. Clasificación de los sistemas microprogramables 43
3.3.1. Microprocesadores 44
3.3.2. Microcontroladores 44
3.3.3. Dispositivos PLDs 44
3.4. Hardware, Software y Firmware 44
3.5. Arquitectura de un sistema basado en CPU 44
3.5.1. Reloj 45
3.5.2. CPU (Unidad Central de Proceso) 45
3.5.2.1. Unidad Aritmético-Lógica (ALU) 45
3.5.2.2. Registros 46
3.5.2.3. Unidad de Control (UC) 46
3.5.3. Memoria 46
3.5.4. Unidad de Entrada/Salida (E/S) 47
3.5.5. Buses del sistema 47

3.5.5.1. Bus de direcciones (*Address Bus*) . . . 47
3.5.5.2. Bus de datos (*Data Bus*) . . . 47
3.5.5.3. Bus de control (*Control Bus*) . . . 47
3.5.5.4. Operaciones de lectura y escritura . . . 47
3.6. Arquitecturas Von Neumann y Harvard . . . 48
3.6.1. Arquitectura Von Neumann . . . 48
3.6.2. Arquitectura Harvard . . . 48
3.7. Software: instrucciones y lenguajes de programación . . . 49
3.7.1. Formato de una instrucción . . . 49
3.7.2. Tipos de instrucciones . . . 50
3.7.3. Lenguajes de programación . . . 50
3.7.3.1. Lenguaje máquina (código máquina) . . . 50
3.7.3.2. Lenguaje ensamblador (*Assembler*) . . . 50
3.7.3.3. Lenguajes de alto nivel . . . 50
3.8. El ATmega328 en este curso . . . 52

4. Pensamiento Computacional y Algoritmos **54**
4.1. ¿Qué es un algoritmo? . . . 54
4.2. Representación de algoritmos: el diagrama de flujo . . . 54
4.2.1. Símbolos básicos . . . 55
4.2.2. Reglas generales . . . 55
4.3. Estructuras de control . . . 55
4.3.1. Secuencia . . . 55
4.3.2. Decisión (selección) . . . 56
4.3.3. Repetición (iteración) . . . 57
4.3.3.1. Repetir mientras (while — condición al inicio) . . . 57
4.3.3.2. Repetir hasta (do-while — condición al final) . . . 58
4.4. Variables . . . 59
4.4.1. Tipos de datos básicos . . . 60
4.4.2. Asignación . . . 60
4.5. Operadores . . . 60
4.5.1. Aritméticos . . . 60
4.5.2. Relacionales (comparación) . . . 60
4.5.3. Lógicos . . . 61
4.6. Herramienta: PSeInt . . . 61
4.6.1. Configuración del perfil . . . 61
4.6.2. Editar el diagrama de flujo . . . 61
4.6.3. Ejecutar el algoritmo . . . 61
4.7. Ejercicios . . . 61
4.7.1. Ejercicio 1 — Suma de dos valores . . . 62
4.7.2. Ejercicio 2 — Operación con tres valores . . . 62
4.7.3. Ejercicio 3 — Módulo de un valor . . . 62
4.7.4. Ejercicio 4 — Sensor de temperatura . . . 62
4.7.5. Ejercicio 5 — Promedio de una serie de valores . . . 62

5. Introducción al Lenguaje C **64**
5.1. Del diagrama de flujo al código . . . 64
5.2. Estructura de un programa en C . . . 64
5.3. Variables y tipos de datos . . . 65
5.3.1. Tipos de datos básicos . . . 65
5.3.2. Ejemplos de declaración . . . 65
5.3.3. Reglas para nombrar variables . . . 66
5.4. Operadores . . . 66
5.4.1. Aritméticos . . . 66
5.4.2. Relacionales . . . 66
5.4.3. Lógicos . . . 67
5.4.4. Asignación . . . 67
5.5. Entrada y salida por consola . . . 67
5.5.1. `printf` — mostrar datos en pantalla . . . 67
5.5.2. `scanf` — leer datos del teclado . . . 67
5.6. Estructuras de control . . . 68
5.6.1. Secuencia . . . 68
5.6.2. Decisión: `if` / `else` . . . 68
5.6.3. Repetición: `while` . . . 68
5.6.4. Repetición: `do-while` . . . 69
5.6.5. Repetición: `for` . . . 69
5.7. Funciones . . . 70
5.7.1. Prototipo y definición . . . 70
5.7.2. Partes de una función . . . 71
5.7.3. Funciones que no devuelven nada: `void` . . . 71
5.7.4. Ejemplo completo . . . 71
5.8. Constantes y macros: `#define` . . . 72
5.8.1. Constantes . . . 72
5.8.2. Macros con parámetros . . . 73
5.9. Equivalencia entre pseudocódigo y C . . . 73
5.10. Ejercicios: los algoritmos del TP en C . . . 73
5.11. Entorno de desarrollo . . . 74

6. Programación de Microcontroladores en C **75**
6.1. C estándar vs. C embebido . . . 75
6.1.1. Estructura básica de un programa embebido . . . 75
6.2. Los puertos de E/S del ATmega328 . . . 76
6.3. Operadores de manejo de bits . . . 77
6.3.1. Desplazamiento de bits . . . 78
6.4. Técnicas de enmascarado . . . 78
6.4.1. Poner un bit en 1 (SET) . . . 78
6.4.2. Poner un bit en 0 (CLEAR) . . . 78
6.4.3. Invertir un bit (TOGGLE) . . . 79
6.4.4. Leer el estado de un bit . . . 79

6.4.5. Macros para manejar bits . . . 79
6.4.5.1. Leer entradas: `bit_is_set` y `bit_is_clear` . . . 79
6.5. Configuración de puertos (TP1) . . . 80
6.5.1. Configurar un pin como salida . . . 80
6.5.2. Configurar un pin como entrada . . . 80
6.5.3. Activar la resistencia de pull-up interna . . . 80
6.5.4. Escribir en un pin de salida . . . 81
6.5.5. Leer un pin de entrada . . . 81
6.5.6. Ejemplo completo — TP1: un pulsador controla un LED . . . 81
6.6. Retardos . . . 82
6.7. Escribir en el puerto completo . . . 82
6.7.1. Escribir solo algunos bits: enmascarar . . . 83
6.8. Display de 7 segmentos (TP4) . . . 83
6.8.1. Conectar el display: los segmentos en dos puertos . . . 85
6.9. Timers e interrupciones (TP5) . . . 86
6.9.1. El problema del código bloqueante . . . 86
6.9.2. Interrupciones . . . 86
6.9.3. Vectores de interrupción del ATmega328 . . . 87
6.9.4. Configuración del Timer 0 en modo CTC . . . 87
6.9.5. Ejemplo — parpadeo de LED sin código bloqueante . . . 88
6.10. Multiplexado de displays (TP6) . . . 89
6.10.1. Conexión de los displays . . . 89
6.10.1.1. Cátodo común: con transistor NPN . . . 89
6.10.1.2. Opción: ánodo común conectado directo al micro . . . 90
6.10.1.3. ¿Con transistor o directo? Diferencias, ventajas y problemas 91
6.10.2. Principio de funcionamiento . . . 92
6.10.3. Implementación con Timer 0 . . . 92
6.11. Eliminación de rebote por software . . . 94
6.11.1. Método bloqueante (simple, para aprender) . . . 94
6.11.2. Método por detección de flanco (no bloqueante, TP5) . . . 94
6.12. Resumen de registros del ATmega328 usados en los TPs . . . 95

Capítulo 1

Lógica Secuencial

1.1. Lógica combinacional vs. lógica secuencial

En lógica combinacional, las salidas dependen únicamente del valor actual de las entradas. En lógica **secuencial**, las salidas dependen tanto de las entradas actuales como del **estado anterior** del circuito. Esto significa que el circuito tiene memoria: recuerda lo que ocurrió antes.

El bloque fundamental de la lógica secuencial es el **dispositivo biestable**: un circuito que puede permanecer indefinidamente en uno de dos estados estables (SET o RESET), y que solo cambia de estado cuando se lo indica una señal de entrada.

Los dispositivos biestables se dividen en dos categorías:

- **Latches:** cambian de estado de forma asíncrona, es decir, responden directamente al nivel de sus entradas en cualquier momento.
- **Flip-flops:** cambian de estado de forma síncrona, es decir, solo en el momento en que ocurre un flanco de la señal de reloj.

1.2. Latches

1.2.1. Latch SR (Set-Reset)

Es el tipo más simple de dispositivo biestable. Tiene dos entradas (S y R) y dos salidas complementarias (Q y $\overline{Q}$).

Puede implementarse con dos compuertas NOR (entradas activas a nivel alto) o con dos compuertas NAND (entradas activas a nivel bajo).

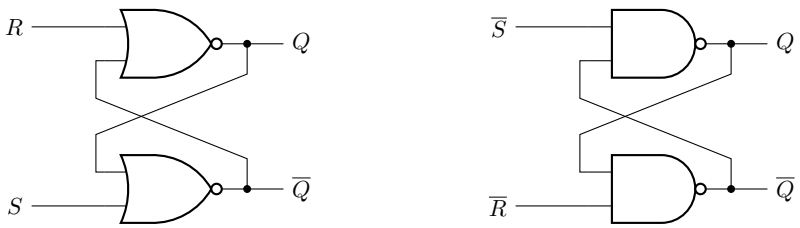


Figura 1.1: Latch SR: implementación con compuertas NOR (izquierda, entradas activas alto) y compuertas NAND (derecha, entradas activas bajo). Los cruces de cables sin punto indican que los conductores no se conectan.

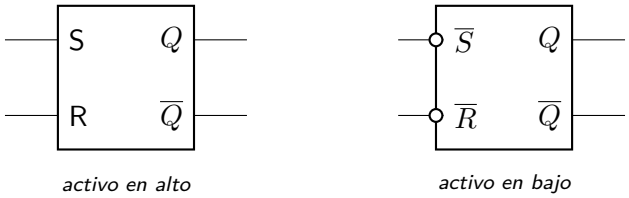


Figura 1.2: Símbolo lógico del latch SR: representación en bloque con entradas S, R y salidas Q, $\overline{Q}$ (figura 7.4 Floyd)

Tabla de verdad — Latch SR (compuertas NOR, entradas activas alto):

S	R	Q	$\overline{Q}$	Condición
0	0	Q_0	$\overline{Q_0}$	No cambia (retiene estado)
1	0	1	0	SET (Q pasa a 1)
0	1	0	1	RESET (Q pasa a 0)
1	1	?	?	**No válido**

La condición no válida ocurre cuando ambas entradas se activan simultáneamente: fuerza a ambas salidas al mismo nivel, violando la condición de complementariedad. Si luego se desactivan al mismo tiempo, el estado final es impredecible.

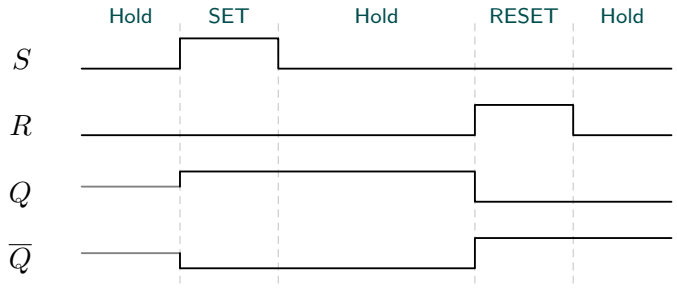


Figura 1.3: Diagrama de tiempos del latch SR (compuertas NOR, entradas activas alto). La franja *Hold* indica que el estado se retiene sin cambios.

Aplicación — Eliminación de rebote de pulsadores:

¿Qué es el rebote de los contactos?

Cuando se presiona un pulsador o se acciona un interruptor mecánico, uno esperaría que la señal eléctrica pasara limpiamente de 0 a 1 en un único instante. Sin embargo, los contactos metálicos son físicamente elásticos: al cerrarse, rebotan varias veces antes de quedar en contacto estable. Lo mismo ocurre al abrir el circuito.

Este fenómeno se llama **rebote de contactos** (*contact bounce* o *switch bounce*). La duración del rebote depende del tipo de pulsador y de su estado mecánico, pero típicamente está entre **1 ms y 20 ms**.

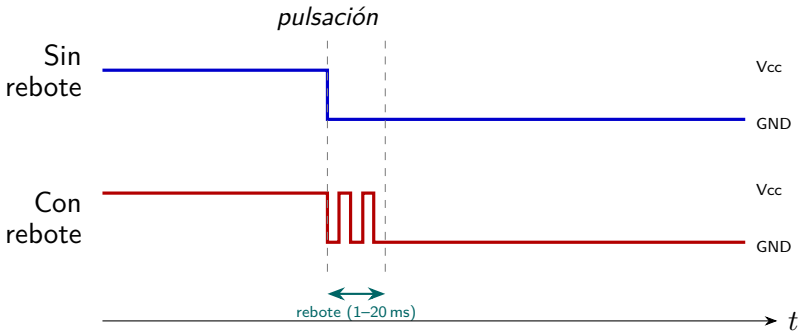


Figura 1.4: Señal ideal (flanco limpio) vs. señal real con rebote (múltiples pulsos breves en la zona de transición)

¿Qué ve el circuito digital durante el rebote?

Para la electrónica digital no existe el concepto de "está rebotando": cada transición de tensión es leída como un cambio de estado válido. Así, un único pulso de dedo puede generar **5, 10 o más pulsos eléctricos** en los primeros milisegundos. La señal real se ve como una ráfaga de pulsos muy cortos, seguida por un nivel estable. Desde el punto de vista del circuito, esos pulsos son completamente válidos e indistinguibles de pulsaciones reales del usuario.

Problemas que genera el rebote

Situación	Consecuencia del rebote
Contador incrementado por botón	Un solo toque incrementa el contador en 3, 7 u otro número imprevisible
Interrupción por flanco en microcontrolador	La ISR se ejecuta múltiples veces por una única pulsación
Latch o flip-flop activado por botón	El estado puede cambiar varias veces antes de estabilizarse
Sistema de navegación de menú	Un toque avanza varios ítems en lugar de uno

Cuadro 1.1: Problemas típicos causados por el rebote de contactos en sistemas digitales.

El rebote es especialmente problemático con señales de reloj o con pines de interrupción, donde el sistema es sensible a cada flanco individualmente.

Solución con latch SR

Un latch SR puede eliminar el rebote mecánico usando un interruptor SPDT (*Single Pole Double Throw*) conectado entre masa y las entradas $\overline{S}$ y $\overline{R}$ del latch, con resistencias pull-up a Vcc.

Al cambiar la posición del interruptor, el primer contacto activa el latch de forma definitiva. Cualquier rebote posterior aplica la misma entrada (S o R) que ya fue procesada, por lo que

el estado del latch no vuelve a cambiar. La salida Q presenta una transición limpia y única.

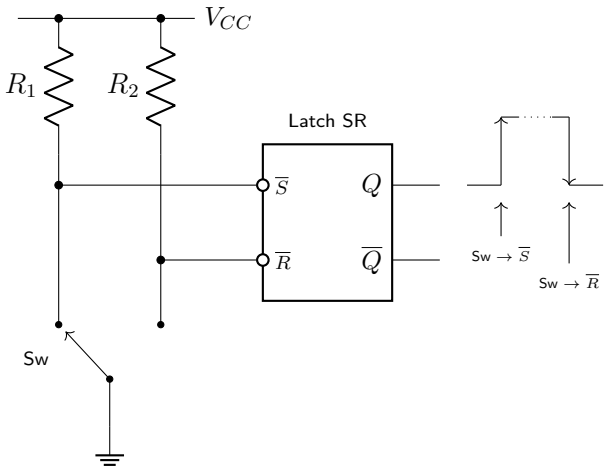


Figura 1.5: Circuito eliminador de rebote de pulsadores con latch SR (figura 7.6 Floyd)

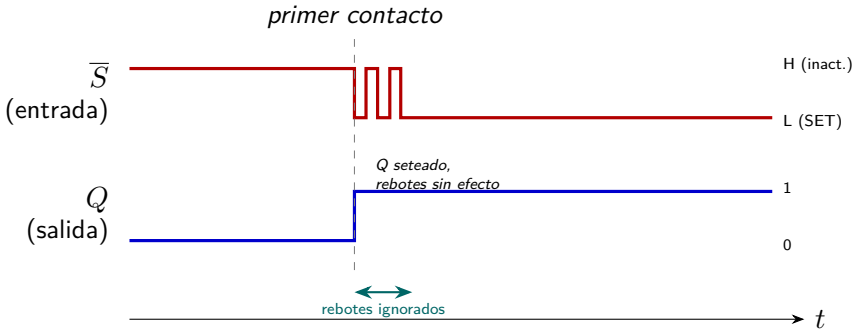


Figura 1.6: Diagrama de tiempos del circuito antirrebote: señal con rebote en la entrada vs. salida Q limpia del latch (figura 7.7 Floyd)

1.2.2. Latch SR con habilitación

Agrega una entrada de habilitación (EN). El latch solo puede cambiar de estado cuando EN está activo. Permite controlar exactamente en qué momento el latch responde a sus entradas.

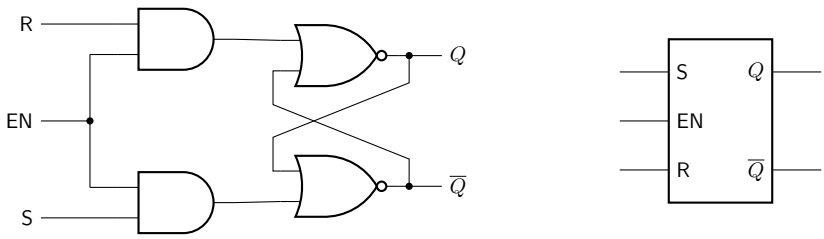


Figura 1.7: Diagrama y símbolo del latch SR con habilitación (EN) (figura 7.8 Floyd)

Tabla de verdad — Latch SR con habilitación (compuertas NOR, entradas activas alto):

S	R	EN	Q	Condición
0	0	1	Q_0	No cambia
1	0	1	1	SET (Q pasa a 1)
0	1	1	0	RESET (Q pasa a 0)
1	1	1	?	**No válido**
X	X	0	Q_0	No cambia (EN inactivo, S y R ignoradas)

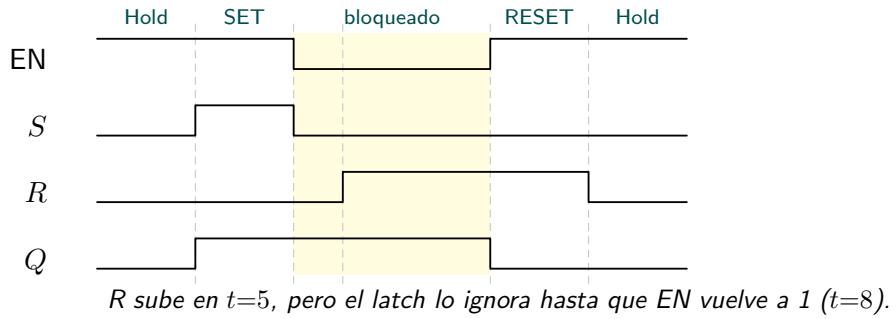


Figura 1.8: Diagrama de tiempos del latch SR con habilitación. EN=0 bloquea las entradas: aunque R se active mientras EN=0, Q no cambia. El RESET recién ocurre cuando EN regresa a 1 (y R sigue activo).

1.2.3. Latch D con habilitación

Tiene una única entrada de datos (D) además de la habilitación (EN). Elimina la condición no válida del latch SR: cuando EN está activo, la salida Q sigue al valor de D. Cuando EN se desactiva, el latch retiene el último valor de D.

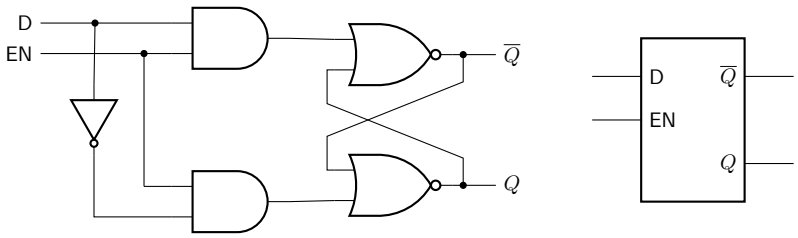


Figura 1.9: Diagrama y símbolo del latch D con habilitación (figura 7.10 Floyd)

D	EN	Q	Condición
0	1	0	RESET
1	1	1	SET
X	0	Q_0	No cambia

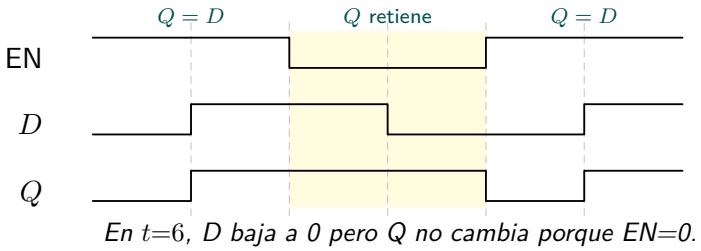


Figura 1.10: Diagrama de tiempos del latch D. Con EN=1 la salida es *transparente*: Q sigue inmediatamente a D. Con EN=0 el latch queda *opaco*: retiene el último valor de D aunque la entrada cambie (zona sombreada; ver $t{=}6$).

1.3. Flip-flops

La diferencia fundamental entre un latch y un flip-flop es el mecanismo de disparo. Un flip-flop **disparado por flanco** solo cambia de estado en el instante exacto del flanco (de subida o de bajada) de la señal de reloj. Fuera de ese instante, es insensible a los cambios en sus entradas.

En los símbolos lógicos, el flanco activo se indica con un **triángulo** en la entrada de reloj (indicador de entrada dinámica). Si hay un círculo adicional, el disparo es por flanco negativo (bajada); sin círculo, por flanco positivo (subida).

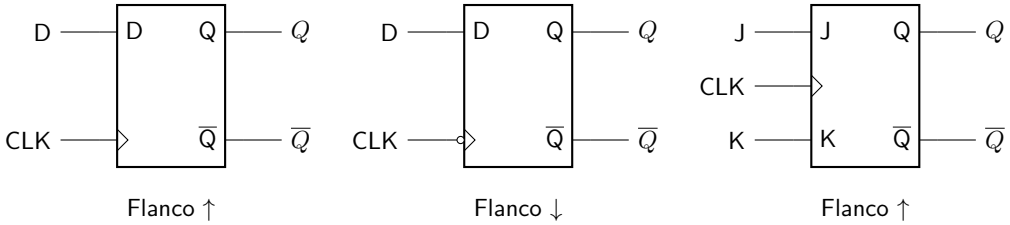


Figura 1.11: Símbolos de flip-flops disparados por flanco. El triángulo en CLK indica disparo por flanco; el círculo adicional (centro) indica flanco negativo.

1.3.1. Flip-flop SR disparado por flanco

Similar al latch SR pero sincronizado con el reloj. Las entradas S y R solo son evaluadas en el flanco activo del reloj.

S	R	Q siguiente	Condición
0	0	Q_0	No cambia
1	0	1	SET
0	1	0	RESET
1	1	?	No válido

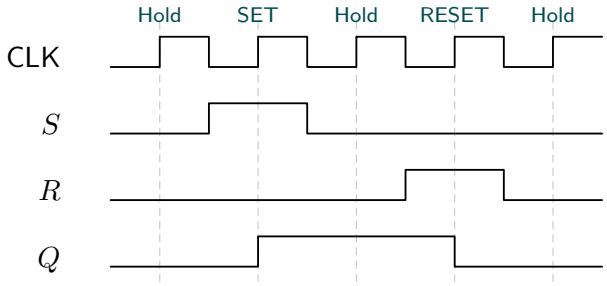


Figura 1.12: Diagrama de tiempos del flip-flop SR disparado por flanco positivo. S y R solo son evaluadas en el flanco de subida del reloj; fuera de ese instante los cambios en las entradas no tienen efecto.

1.3.2. Flip-flop D disparado por flanco

La salida Q toma el valor de la entrada D en cada flanco activo del reloj. Es el más simple y el más utilizado en la práctica.

D	Q siguiente
0	0
1	1

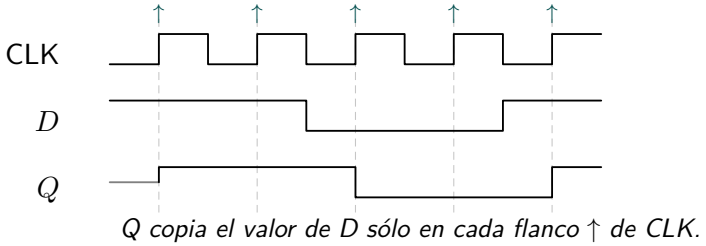


Figura 1.13: Diagrama de tiempos del flip-flop D disparado por flanco positivo. Cada flanco de subida del reloj transfiere el valor de D a la salida Q.

1.3.3. Flip-flop JK disparado por flanco

El flip-flop JK resuelve la condición no válida del SR reemplazando el estado indeterminado por una operación de **basculación (toggle)**: cuando J = K = 1, la salida cambia de estado en cada flanco de reloj.

J	K	Q siguiente	Condición
0	0	Q_0	No cambia
1	0	1	SET
0	1	0	RESET
1	1	$\overline{Q_0}$	Bascula (Toggle)

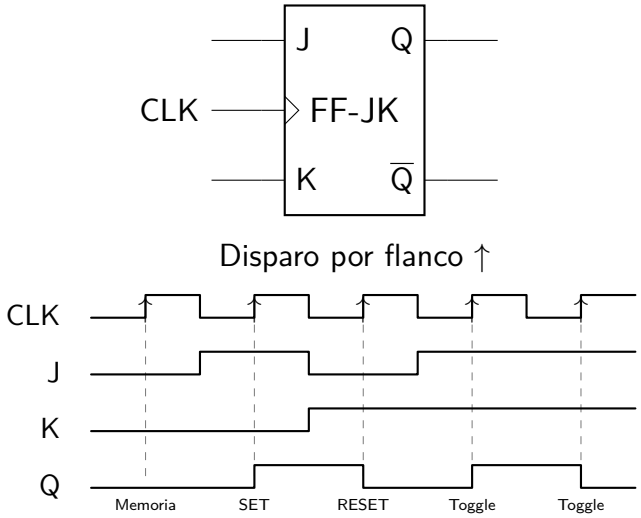


Figura 1.14: Símbolo del flip-flop JK disparado por flanco (figura 7.22 Floyd)

1.3.4. Flip-flop T (Toggle)

Es un caso especial del JK con ambas entradas unidas (J = K = T). Cuando T = 1, la salida bascula en cada flanco de reloj. Cuando T = 0, la salida no cambia.

T	Q siguiente
0	Q_0
1	$\overline{Q_0}$

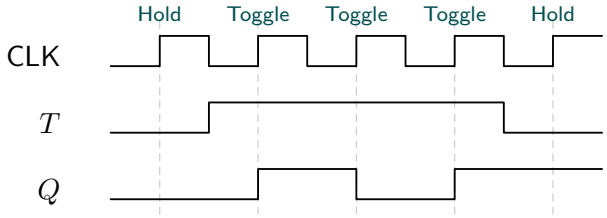


Figura 1.15: Diagrama de tiempos del flip-flop T. Con T=0 la salida no cambia en el flanco activo; con T=1 la salida bascula en cada flanco de subida, dividiendo la frecuencia del reloj por 2.

El flip-flop T es la base de los contadores binarios.

1.3.5. Entradas asíncronas: Preset y Clear

La mayoría de los flip-flops tienen entradas asíncronas que actúan **independientemente del reloj**:

- **Preset (PRE):** fuerza la salida Q a 1.
- **Clear (CLR):** fuerza la salida Q a 0.

Son útiles para inicializar el sistema a un estado conocido al encender, o para hacer resets de emergencia.

1.3.6. Características de funcionamiento

Algunas características importantes de los flip-flops al trabajar con señales reales:

Retardo de propagación (t_{pd}): tiempo que transcurre desde el flanco activo del reloj hasta que la salida estabiliza su nuevo valor. Limita la frecuencia máxima de operación.

Tiempo de establecimiento (t_{setup}): tiempo mínimo que la entrada debe estar estable **antes** del flanco activo del reloj para garantizar una lectura correcta.

Tiempo de mantenimiento (t_{hold}): tiempo mínimo que la entrada debe permanecer estable **después** del flanco activo del reloj.

Frecuencia máxima (f_{max}): frecuencia de reloj máxima a la que el flip-flop puede operar correctamente.

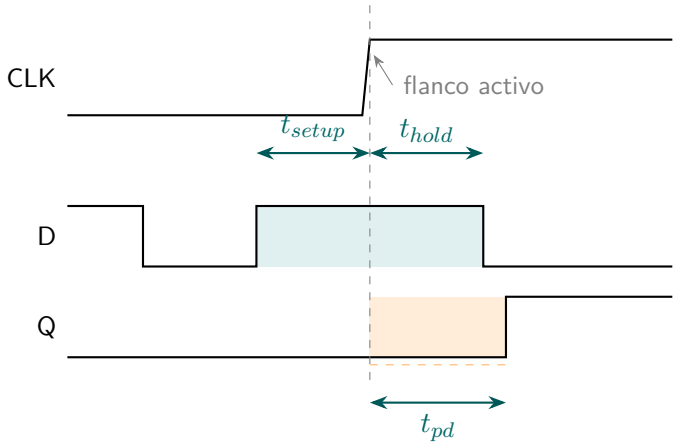


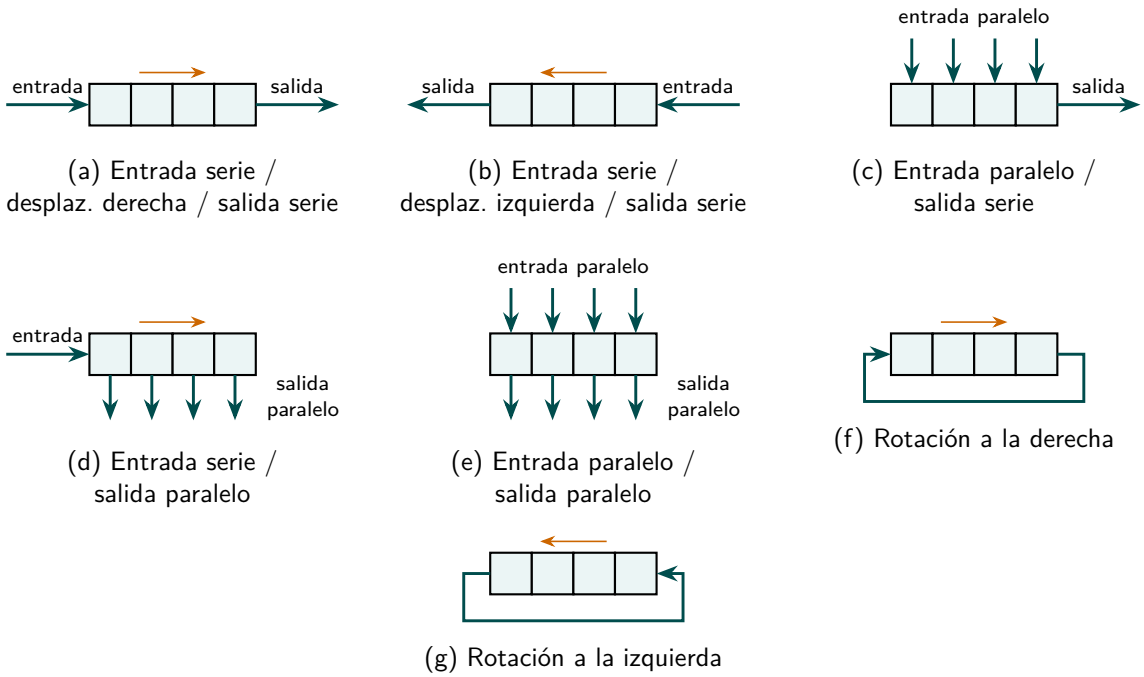
Figura 1.16: Parámetros temporales de un flip-flop: t_{setup} (D debe estar estable antes del flanco), t_{hold} (D debe mantenerse estable después del flanco), t_{pd} (tiempo desde el flanco hasta que Q es válido). La zona sombreada en D indica la ventana de estabilidad requerida.

1.4. Registros de desplazamiento

Un registro de desplazamiento es un circuito formado por flip-flops en cascada que cumple dos funciones: **almacenamiento** y **desplazamiento** de datos. Cada flip-flop representa un

bit; por lo tanto, un registro de N etapas almacena N bits.

Los datos pueden entrar y salir del registro de distintas formas:



(a) Entrada serie / desplaz. derecha / salida serie

(b) Entrada serie / desplaz. izquierda / salida serie

(c) Entrada paralelo / salida serie

(d) Entrada serie / salida paralelo

(e) Entrada paralelo / salida paralelo

(f) Rotación a la derecha

(g) Rotación a la izquierda

Figura 1.17: Los 7 modos básicos de movimiento de datos en un registro de 4 bits (figura 9.2 Floyd)

1.4.1. Entrada serie / Salida serie (SISO)

Los bits entran uno a uno por un extremo y salen uno a uno por el otro. En cada pulso de reloj, todos los bits se desplazan una posición. Se necesitan N pulsos para cargar N bits y otros N para extraerlos.

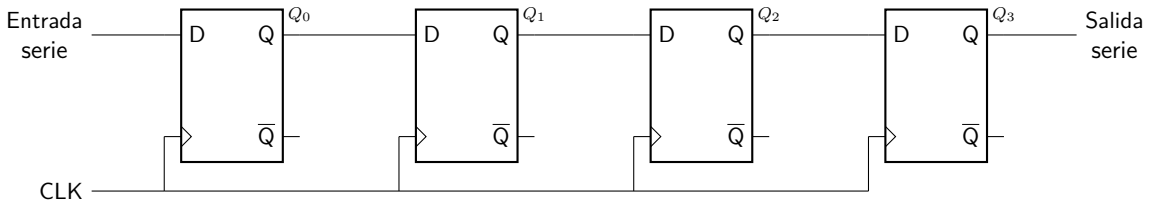


Figura 1.18: Registro de desplazamiento SISO de 4 bits: flip-flops D en cascada (figura 9.3 Floyd)

Nota

CI típico: 74HC164 — registro SIPO de 8 bits (serie in / paralelo out). También puede usarse como SISO tomando solo la salida del último flip-flop.

1.4.2. Entrada serie / Salida paralelo (SIPO)

Los bits entran en serie pero se leen simultáneamente de todas las salidas Q de cada etapa. Una vez cargados los N bits, están disponibles todos al mismo tiempo en las salidas paralelo. Se usa para convertir datos serie a paralelo, por ejemplo para controlar múltiples LEDs o segmentos usando solo unos pocos pines del microcontrolador.

Nota

CI típicos:

- **74HC595** — registro SIPO de 8 bits con latch de salida separado y una salida serie adicional ($Q_{H'}$) que permite conectar varios en cascada. Es el más utilizado en proyectos con microcontroladores para expandir salidas digitales.
- **74HC164** — registro SIPO de 8 bits sin latch de salida (las salidas cambian mientras se desplazan los datos).

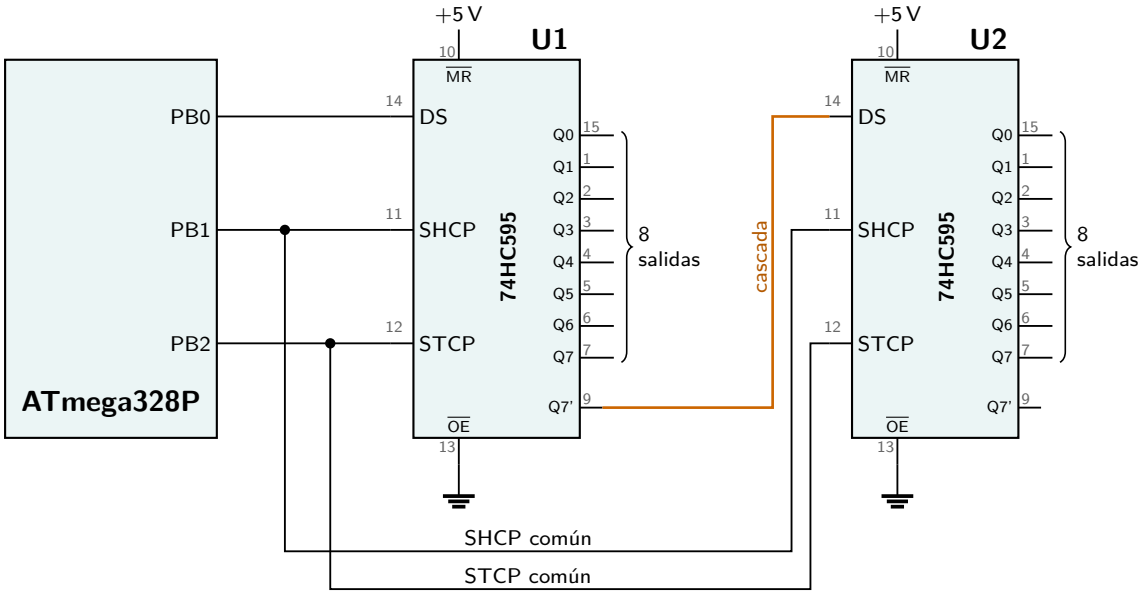


Figura 1.19: Conexión de dos 74HC595 en cascada a un ATmega328P: 3 pines del micro controlan 16 salidas

1.4.3. Entrada paralelo / Salida serie (PISO)

Los N bits se cargan simultáneamente en las etapas (carga paralelo) y luego se extraen uno a uno en serie. Se usa para convertir datos paralelo a serie, por ejemplo para leer el estado de múltiples entradas digitales usando pocos pines del microcontrolador.

Nota

CI típicos:

- **74HC165** — registro PISO de 8 bits con carga paralela asíncrona. Muy usado para leer matrices de pulsadores o DIP switches desde un microcontrolador por SPI.
- **74LS165** — versión TTL del mismo.

1.4.4. Entrada paralelo / Salida paralelo (PIPO)

Los datos se cargan y se leen en paralelo. Funciona esencialmente como un banco de flip-flops D con habilitación. Se usa principalmente como registro de almacenamiento temporal o buffer de datos.

Nota

CI típico: 74HC195 — registro PIPO de 4 bits con entrada serie y paralelo y salida paralelo.

1.4.5. Registros bidireccionales

Pueden desplazar datos tanto hacia la izquierda como hacia la derecha, según el valor de una entrada de control de dirección. Son los más versátiles ya que combinan múltiples modos de operación.

Nota

CI típicos:

- **74HC194** — registro universal de 4 bits: SISO izquierda, SISO derecha, carga paralelo, retención. Controlado por dos entradas de modo (S0, S1).
- **74LS194** — versión TTL del mismo.

1.4.6. Contadores basados en registros de desplazamiento

Conectando la salida del último flip-flop a la entrada del primero (registro en anillo), se obtienen secuencias especiales:

Contador en anillo: desplaza un único bit activo de forma circular. Un registro de N etapas genera una secuencia de N estados únicos y requiere N flip-flops para contar N estados (sin decodificador adicional).

Contador Johnson (o de anillo torcido): la salida complementada del último flip-flop ($\overline{Q}$) se realimenta a la entrada del primero. Un registro de N etapas genera una secuencia de 2N estados.

1.5. Contadores

Un contador es un circuito secuencial formado por flip-flops que recorre una secuencia determinada de estados binarios, uno por cada pulso de reloj. El **módulo** de un contador es la cantidad total de estados distintos que recorre antes de repetirse.

$$\text{Módulo} = 2^N \quad \text{(para un contador binario de N flip-flops)}$$

1.5.1. Contadores asíncronos (con propagación)

El reloj se aplica solo al primer flip-flop. Cada flip-flop siguiente se dispara con la salida del anterior. Los cambios de estado se propagan de etapa en etapa, por eso también se los llama **contadores con propagación** (*ripple counters*).

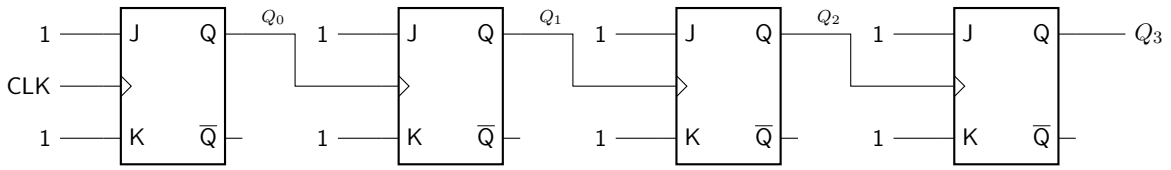


Figura 1.20: Contador asíncrono (ripple) de 4 bits con flip-flops JK en modo toggle. La salida Q de cada FF actúa como reloj del siguiente (figura 8.1 Floyd).

Diagrama de tiempos de un contador asíncrono de 3 bits:

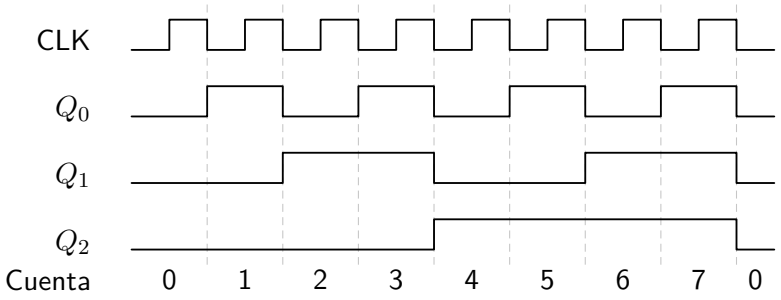


Figura 1.21: Diagrama de tiempos del contador asíncrono de 3 bits (módulo 8). Q0 es el bit de menor peso (LSB) y Q2 el de mayor peso (MSB). Cada salida divide por 2 la frecuencia de la señal anterior.

La secuencia de salidas Q2 Q1 Q0 va de 000 a 111 (0 a 7), y luego vuelve a 000. Esto es un contador módulo-8.

Desventaja: el retardo de propagación se acumula etapa a etapa. En contadores muy rápidos o con muchas etapas, los estados intermedios falsos (*glitches*) pueden causar problemas en la lógica de decodificación.

Contador de décadas asíncrono: un contador que llega solo hasta 9 (BCD) y luego se reinicia. Se obtiene forzando el reset del contador cuando se alcanza la combinación binaria 10 (1010_2).

1.5.2. Contadores síncronos

El reloj se conecta **simultáneamente** a todos los flip-flops. Los cambios de estado ocurren al mismo tiempo en todas las etapas, eliminando el problema de acumulación de retardos.

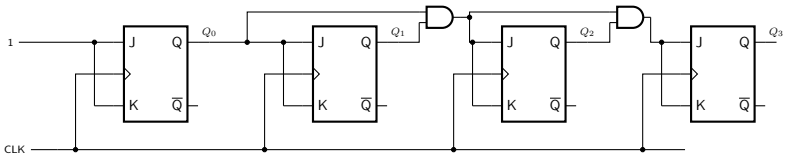


Figura 1.22: Contador síncrono de 4 bits con flip-flops JK (figura 8.16 Floyd)

La lógica de habilitación de cada etapa se diseña para que el flip-flop cambie de estado solo cuando todos los bits de menor peso estén en 1 (condición de acarreo).

Ventajas sobre los asíncronos:

- Todos los bits cambian simultáneamente → no hay glitches por propagación.
- Pueden operar a mayor frecuencia.
- Son preferidos en diseños digitales modernos.

1.5.3. Contadores ascendentes/descendentes

Un contador puede recorrer su secuencia en sentido creciente (ascendente, *up*) o decreciente (descendente, *down*). Con una entrada de control de dirección se puede elegir el sentido en tiempo real.

1.5.4. Módulo de un contador y truncamiento de secuencia

Un contador de N flip-flops tiene módulo 2^N por defecto. Para obtener un módulo M menor, se detecta el estado M (cuando el contador llega a M) y se fuerza un reset síncrono o asíncrono, haciendo que el contador salte desde M-1 de vuelta a 0.

Ejemplo: contador módulo-5 con 3 flip-flops (que tendrían módulo 8 por defecto). Se detecta el estado 5 (101_2) y se usa para resetear el contador, obteniendo la secuencia 0→1→2→3→4→0→...

1.5.5. Contadores en cascada

Se pueden conectar varios contadores en serie para obtener módulos más grandes. La señal de fin de cuenta (*terminal count*, TC) del primer contador habilita el reloj del segundo.

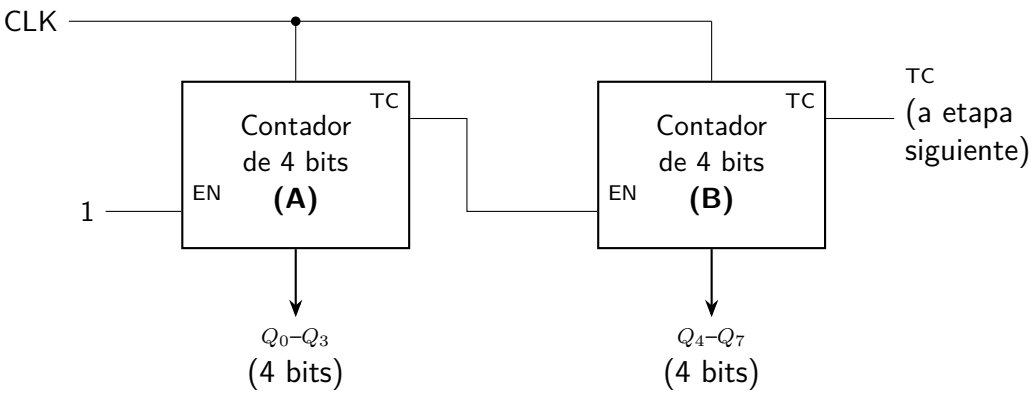


Figura 1.23: Dos contadores de 4 bits en cascada formando un contador de 8 bits (figura 8.38 Floyd)

1.5.6. Decodificación de estados

Para detectar un estado específico del contador se puede usar una compuerta AND que active su salida solo cuando las salidas del contador forman la combinación deseada. Se debe tener cuidado con los *glitches* en contadores asíncronos.

1.6. Máquinas de estado

Una máquina de estado finito (FSM, *Finite State Machine*) es un modelo para describir sistemas secuenciales de forma sistemática. Se define por:

- Un conjunto de **estados** posibles.
- Un **estado inicial**.
- Las **condiciones de transición** entre estados (dependientes de las entradas).
- Las **salidas** asociadas a cada estado o a cada transición.

1.6.1. Diagrama de estados

El comportamiento de una máquina de estados se representa gráficamente con un **diagrama de estados**:

- Cada estado se representa con un círculo o nodo.
- Las transiciones entre estados se representan con flechas etiquetadas con la condición de entrada que las provoca.
- Las salidas se indican dentro del nodo (Moore) o sobre la flecha (Mealy).

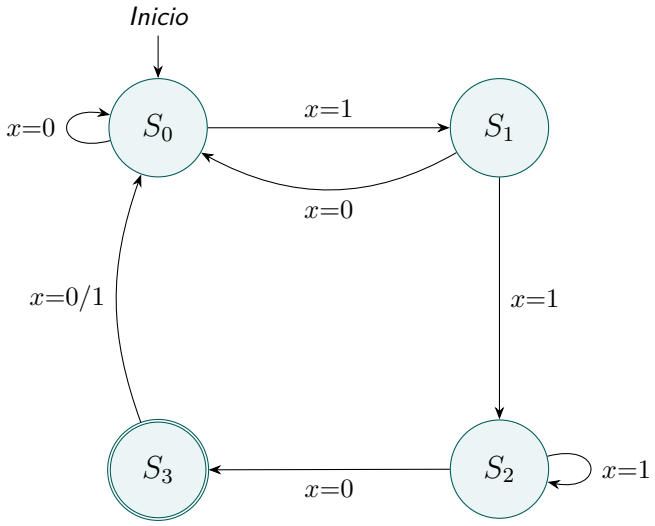


Figura 1.24: Ejemplo de diagrama de estados (FSM de Moore con 4 estados). El estado inicial S_0 tiene doble flecha de entrada; S_3 es el estado de aceptación (doble círculo). Las etiquetas sobre las flechas indican la condición de entrada que provoca la transición.

1.6.2. Máquina de Moore

Las salidas dependen **únicamente del estado actual**. Son más fáciles de diseñar y menos propensas a glitches en las salidas.

1.6.3. Máquina de Mealy

Las salidas dependen del **estado actual y de las entradas**. Pueden reaccionar más rápido a los cambios de entrada, pero las salidas pueden tener transitorios.

1.6.4. Tabla de estados

Una forma alternativa y más formal de describir una máquina de estados es mediante una tabla:

Estado actual	Entrada	Estado siguiente	Salida
S0	0	S0	0
S0	1	S1	0
S1	0	S0	1
S1	1	S2	0
...	...	...	...

1.6.5. Proceso de diseño de una máquina de estados

1. Definir el comportamiento deseado y los estados necesarios.
2. Elaborar el diagrama de estados.

3. Asignar una codificación binaria a cada estado.
4. Derivar las ecuaciones de estado siguiente y de salida (usando mapas de Karnaugh si es necesario).
5. Implementar el circuito con flip-flops y lógica combinacional.

1.7. Actividades

Las siguientes actividades parten de situaciones concretas: primero se describe un problema real y recién después se pide la solución técnica. En todos los casos conviene responder en este orden: entender qué tiene que *recordar* el circuito, elegir el dispositivo adecuado (latch, flip-flop, registro o contador) y recién entonces dibujar.

1.7.1. Latches

Actividad 1 — La puerta que nadie vio abrirse

En un depósito hay una puerta de emergencia con un sensor magnético que entrega **1** mientras la puerta está abierta y **0** cuando está cerrada. El encargado sospecha que alguien la abre de noche, pero a la mañana siempre la encuentra cerrada: el sensor ya volvió a 0 y no queda ningún rastro del evento. Se necesita un circuito que "recuerde" que la puerta estuvo abierta, y que solo pueda borrarse con un pulsador de reconocimiento al que accede únicamente el encargado.

a) Explicar por qué conectar un LED directamente al sensor no resuelve el problema. ¿Qué tipo de lógica hace falta?

b) Diseñar el circuito con un latch SR de compuertas NOR. Indicar qué señal se conecta a S y cuál a R, y qué significa $Q = 1$.

c) Completar la evolución de Q para esta secuencia de eventos: puerta cerrada al iniciar el turno $\rightarrow$ la puerta se abre $\rightarrow$ la puerta se cierra $\rightarrow$ el encargado presiona el pulsador de reconocimiento.

d) ¿Qué ocurre si el encargado mantiene presionado el pulsador de reconocimiento *mientras* la puerta está abierta? Relacionar la respuesta con la condición no válida $S = R = 1$ y proponer una solución.

Actividad 2 — Parada de emergencia que no se rearma sola

Una máquina cortadora de papel tiene un botón hongo de emergencia. Al golpearlo, el motor debe detenerse y **quedar enclavado**: aunque el botón vuelva a su posición de reposo (los hongos se destraban girándolos), la máquina no puede volver a arrancar sola. Solo el operario habilitado, con una llave de reset, puede rearmarla.

a) Si se define $Q = 1$ como "máquina habilitada", indicar qué entrada del latch recibe el botón de emergencia y cuál la llave de reset.

b) Explicar por qué el enclavamiento es un requisito de seguridad y no una comodidad. ¿Qué pasaría, en términos del circuito, si se usara una lógica sin memoria?

c) Las normas de seguridad exigen que ante una falla la máquina quede detenida. Analizar la implementación con compuertas NAND y entradas activas a nivel bajo (con resistencias de pull-up): ¿qué sucede si se **corta** el cable que va al botón de emergencia? Comparar con la implementación NOR, entradas activas a nivel alto.

d) Se agrega un "modo mantenimiento": una llave que impide que el circuito pueda rearmarse, incluso si alguien acciona el reset. Indicar qué entrada del latch cumple esa función y escribir la tabla de verdad completa del circuito resultante.

1.7.2. Flip-flops

Actividad 3 — Una sola frecuencia, varios consumidores

Un equipo tiene un único oscilador de **1 kHz**. Una parte del circuito necesita 500 Hz y otra 250 Hz. Agregar dos osciladores más encarece el diseño y, además, señales de distinto origen no quedan sincronizadas entre sí.

a) Diseñar el circuito con flip-flops T (con T fijo en 1). Indicar cuántas etapas hacen falta y de qué salida se toma cada frecuencia.

b) Dibujar el diagrama de tiempos de CLK, Q_0 y Q_1 para 8 pulsos de reloj, considerando disparo por flanco de subida.

c) Escribir la expresión general de la frecuencia a la salida de la etapa N.

d) El mismo equipo necesita también una señal de **1 Hz** para un reloj de segundos. ¿Se puede obtener con esta misma cadena de flip-flops T? Justificar con números.

Actividad 4 — El sensor que cambia cuando no debe

En una cinta transportadora, un sensor óptico entrega 1 mientras una pieza tapa el haz. El control de la máquina toma decisiones una vez por ciclo, marcado por un pulso de reloj de proceso. El problema: el sensor cambia en cualquier momento (incluso en medio de un cálculo), y el sistema a veces toma dos decisiones distintas sobre la misma pieza.

a) Explicar por qué un latch D con habilitación no resuelve del todo el problema mientras EN está activo. Usar el término *transparente*.

b) Con un flip-flop D disparado por flanco de subida, completar la salida Q para la siguiente secuencia (cada columna es un instante; el flanco de reloj ocurre en las columnas marcadas con ↑):

Instante	1	2↑	3	4↑	5	6↑	7	8↑
D	0	0	1	1	0	1	1	0
Q	0	?	?	?	?	?	?	?

c) El flip-flop elegido especifica $t_{setup} = 20\,\text{ns}$. Si el sensor cambia de estado 5 ns antes del flanco activo, ¿qué se puede afirmar sobre el valor que quedará en Q?

d) *Para pensar:* si el sensor es totalmente asíncrono respecto del reloj, la situación del punto anterior es inevitable cada tanto. ¿Qué ventaja tendría conectar un segundo flip-flop D en cascada, con el mismo reloj, antes de usar la señal?

1.7.3. Registros de desplazamiento

Actividad 5 — Ocho LEDs y solo tres pines libres

Un cartel luminoso lleva 8 LEDs que deben encenderse en patrones arbitrarios. El microcontrolador del proyecto ya tiene casi todos los pines ocupados: quedan libres solamente **3**.

a) Indicar qué modo de registro de desplazamiento resuelve el problema y qué circuito integrado de los vistos conviene usar.

b) Determinar cuántos pulsos de reloj se necesitan para cargar un patrón completo de 8 bits.

c) Para el patrón `10110001` (donde el bit de la izquierda debe quedar en la salida más lejana de la entrada serie), escribir el orden exacto en que hay que presentar los bits en la línea de datos.

d) Explicar qué problema visual aparecería si se usara un 74HC164 en lugar de un 74HC595, y qué función del 595 lo evita.

e) El cartel crece a 16 LEDs. Indicar cuántos pines del microcontrolador se necesitan ahora y cuántos pulsos de reloj por actualización.

Actividad 6 — Ocho llaves y los mismos tres pines

El mismo equipo de la actividad anterior tiene un panel con 8 llaves DIP de configuración que hay que leer al encender. Otra vez, no sobran pines.

a) Indicar qué modo de registro corresponde y qué integrado usar.

b) Describir la secuencia completa de operación, separando la etapa de carga paralela de la etapa de desplazamiento.

c) Determinar cuántos pulsos de reloj hacen falta para leer las 8 llaves y qué bit llega primero al microcontrolador.

d) Si un operario mueve una llave *mientras* se está desplazando la información, ¿qué valor lee el microcontrolador: el viejo o el nuevo? Justificar a partir del momento en que se tomó la carga paralela.

e) Sumando las 8 salidas de la actividad 5 y estas 8 entradas: ¿cuántos pines del microcontrolador se usan en total? Comparar con la conexión directa de los 16 dispositivos.

Actividad 7 — El cartel de la gomería

Un cartel tiene 4 lámparas en fila que deben encenderse de a una, en secuencia circular, para simular movimiento. Más adelante el dueño pide otro efecto: que las lámparas se vayan "llenando" de a una hasta quedar todas encendidas y después se vayan apagando de a una, sin agregar más flip-flops.

a) Escribir la secuencia completa de estados de un contador en anillo de 4 etapas e indicar cuántos estados distintos recorre.

b) Explicar por qué este circuito necesita obligatoriamente inicializarse con las entradas Preset y Clear. ¿Qué pasa si al encender los cuatro flip-flops quedan en 0?

c) Escribir la secuencia completa de estados de un contador Johnson de 4 etapas y verificar que produce el segundo efecto pedido.

d) Comparar las tres alternativas para generar 8 estados de salida distintos: contador en anillo, contador Johnson y contador binario con decodificador. Indicar cuántos flip-flops y cuánta lógica adicional requiere cada una.

1.7.4. Contadores

Actividad 8 — Semáforo peatonal de seis fases

Un cruce escolar tiene un semáforo cuyo ciclo se compone de 6 fases que se repiten indefinidamente (verde vehicular, amarillo, rojo, cruce peatonal, peatonal intermitente, rojo total). Cada fase se mantiene un tiempo fijo, marcado por un pulso de reloj.

a) Determinar cuántos flip-flops se necesitan y cuál sería el módulo natural de ese contador.

b) Indicar qué estado hay que detectar para truncar la secuencia y con qué compuerta se implementa esa detección.

c) Escribir la secuencia de estados resultante y verificar que el módulo sea 6.

d) Con reset asíncrono, el estado de truncamiento aparece durante un instante muy breve antes de borrarse. Explicar qué problema puede causar eso en la salida del semáforo y cómo lo evita un reset síncrono.

Actividad 9 — El contador de segundos de un reloj

Se quiere construir la parte de segundos de un reloj digital: dos dígitos que cuenten de 00 a 59 y vuelvan a 00, a partir de una señal de 1 Hz.

a) Indicar el módulo que debe tener el contador de unidades y el de decenas.

b) Explicar cómo se conecta la señal de fin de cuenta (*terminal count*) del contador de unidades para que las decenas avancen en el momento correcto.

c) Verificar que el módulo total del conjunto sea 60 y explicar de dónde sale ese número.

d) ¿Cada cuánto tiempo produce un pulso la salida de fin de cuenta del conjunto completo? ¿Para qué se usaría esa señal en el reloj?

e) Retomando la actividad 3: si la única señal disponible es el oscilador de 1 kHz, indicar qué cadena de contadores hay que agregar para obtener el 1 Hz que necesita este circuito.

Actividad 10 — El ascensor que vuelve a planta baja

Un ascensor de un edificio de 8 pisos (0 a 7) tiene un sensor que entrega un pulso cada vez que pasa frente a un piso, y una señal de dirección que indica si está subiendo o bajando. Un display debe mostrar el piso actual.

a) Determinar cuántos flip-flops se necesitan para representar los 8 pisos.

b) Indicar qué tipo de contador resuelve el problema y qué función cumple la señal de dirección.

c) Durante las pruebas se detecta una falla: si el ascensor llega al piso 7 y recibe un pulso más, el display muestra planta baja. Explicar por qué ocurre y proponer una solución basada en la decodificación del estado 7 junto con la señal de dirección.

d) El piso 5 tiene un cartel que debe encenderse solo cuando el ascensor está en ese piso. Indicar qué compuerta y qué conexiones detectan el estado correspondiente.

1.7.5. Guía de resolución

Nota

Esta subsección es material de apoyo para la corrección. Las respuestas están sintetizadas: se espera que el alumno desarrolle además los diagramas correspondientes.

Actividad 1

a) El LED es lógica combinacional pura: se enciende solo mientras el sensor entrega 1 y se apaga en cuanto la puerta vuelve a cerrarse. El evento "la puerta estuvo abierta" exige recordar algo que ya pasó, es decir, lógica secuencial con un elemento de memoria.

b) El sensor va a S (la apertura pone el latch en SET) y el pulsador de reconocimiento a R. $Q = 1$ significa "hubo una apertura no reconocida". El LED se conecta a Q.

c) Q arranca en 0 (estado borrado). Al abrirse la puerta, $S = 1$ y Q pasa a 1. Al cerrarse, S vuelve a 0 y R sigue en 0: es la condición de retención, por lo que Q **permanece en 1**. Al presionar el pulsador de reconocimiento, $R = 1$ y Q vuelve a 0.

d) Se da $S = R = 1$, la condición no válida: ambas salidas quedan forzadas al mismo nivel y, si las entradas se desactivan casi simultáneamente, el estado final es impredecible. Soluciones posibles: agregar lógica que inhiba el reconocimiento mientras el sensor esté activo (por ejemplo $R' = R \cdot \overline{S}$), o reemplazar el latch SR por un latch D con habilitación, que no tiene condición prohibida.

Actividad 2

a) Con $Q = 1 =$ máquina habilitada: el botón de emergencia va a R (fuerza $Q = 0$) y la llave de reset a S (rearma). El motor se comanda desde Q.

b) Sin memoria, la máquina volvería a arrancar sola apenas se destrabe el botón, con el operario todavía trabajando sobre ella. El enclavamiento garantiza que el rearme sea una decisión humana deliberada y no una consecuencia automática de destrabar el hongo.

c) Con entradas activas a nivel bajo y pull-up, el cable en reposo está en 1 (inactivo) y el botón lo lleva a 0 (activo). Si ese cable se corta, la entrada queda en 1 por el pull-up: el circuito nunca detecta la emergencia. La conclusión es la opuesta de lo que suele suponerse: para que una falla de cableado provoque la **parada** y no la inhiba, la emergencia debe activarse al abrir el circuito (cable cortado $\rightarrow$ la entrada cae a su nivel activo). Este es el criterio de *contacto normalmente cerrado* que usan las normas de seguridad de máquinas.

d) El modo mantenimiento corresponde a la entrada de habilitación EN: con EN inactivo, el latch ignora tanto S como R y retiene su estado (con $Q = 0$, máquina detenida). La tabla es la del latch SR con habilitación: con EN$= 1$, comportamiento normal SET/RESET/retención; con EN$= 0$, $Q = Q_0$ para cualquier valor de S y R.

Actividad 3

a) Dos flip-flops T en cascada, cada uno con $T = 1$, tomando la salida Q de cada etapa como reloj de la siguiente. Q_0 entrega 500 Hz y Q_1 entrega 250 Hz.

b) Q_0 cambia en cada flanco de subida de CLK (período doble que CLK); Q_1 cambia en cada flanco de subida de Q_0 (período cuádruple respecto de CLK). Es el mismo diagrama del contador asíncrono de 3 bits, limitado a dos etapas.

c) $f_N = f_{CLK}/2^N$, siendo N el número de etapas atravesadas.

d) No. Haría falta dividir por 1000, y 1000 no es potencia de 2 ($2^9 = 512$, $2^{10} = 1024$). Una cadena de flip-flops T solo divide por potencias de 2, así que se necesitan contadores con módulo truncado (ver actividad 9).

Actividad 4

a) Mientras EN está activo, el latch D es transparente: Q sigue a D en tiempo real, de modo que cualquier cambio del sensor durante ese intervalo se propaga al control. El latch acota *cuándo* puede pasar la señal, pero no la reduce a un único instante.

b) Q toma el valor de D únicamente en los flancos: instante 2 $\rightarrow Q = 0$; instante 4 $\rightarrow Q = 1$; instante 6 $\rightarrow Q = 1$; instante 8 $\rightarrow Q = 0$. Entre flancos Q se mantiene, de modo que el cambio de D en el instante 3 y el del instante 5 no tienen efecto inmediato.

c) Se viola el tiempo de establecimiento ($5 < 20$ ns), así que no hay garantía: Q puede quedar en el valor nuevo, en el viejo o en un nivel indeterminado durante un lapso. El fabricante solo garantiza el resultado si se respetan t_{setup} y t_{hold}.

d) El primer flip-flop absorbe la violación de t_{setup}; para el flanco siguiente, su salida ya se estabilizó en 0 o en 1, de modo que el segundo flip-flop recibe una entrada estable y entrega al sistema un valor limpio. Se paga un ciclo de reloj de retardo. Es la estructura llamada *sincronizador de dos etapas*, de uso estándar al ingresar cualquier señal asíncrona a un sistema síncrono.

Actividad 5

a) Registro SIPO (entrada serie, salida paralelo), con un **74HC595**. Los tres pines son: datos serie, reloj de desplazamiento y reloj de latch de salida.

b) 8 pulsos de reloj, uno por bit.

c) El primer bit enviado es el que más se desplaza, así que termina en la salida más lejana. Para que el bit de la izquierda quede allí, hay que enviar primero el MSB: 1 – 0 – 1 – 1 – 0 – 0 – 0 – 1.

d) El 74HC164 no tiene latch de salida: sus salidas cambian a medida que los datos se desplazan, por lo que durante la carga se ve un "barrido" de encendidos intermedios que no corresponden al patrón final. El 595 mantiene las salidas congeladas y las actualiza todas juntas al pulso de latch.

e) Siguen siendo **3 pines**: se conectan dos 595 en cascada (la salida serie del primero alimenta la entrada serie del segundo, compartiendo reloj y latch). Ahora hacen falta 16 pulsos de reloj por actualización.

Actividad 6

a) Registro PISO (entrada paralelo, salida serie), con un **74HC165**.

b) Primero se activa la carga paralela: el registro toma simultáneamente el estado de las 8 llaves. Luego se desactiva la carga y se aplican pulsos de reloj para desplazar los bits, uno por pulso, hacia el microcontrolador.

c) 8 pulsos. Llega primero el bit de la etapa más cercana a la salida serie, es decir, el último del registro.

d) Lee el valor **viejo**: la carga paralela funciona como una fotografía tomada en un instante, y el desplazamiento posterior trabaja sobre esa copia. El cambio recién aparecerá en la lectura siguiente. Esta propiedad es deseable: garantiza que los 8 bits leídos correspondan todos al mismo instante.

e) Se pueden compartir el reloj y usar líneas de datos separadas: con 5 pines ($2\times$ datos, reloj compartido, latch del 595, carga del 165) se manejan los 16 dispositivos, frente a los 16 pines que exigiría la conexión directa. Incluso sin compartir nada, 6 pines contra 16.

Actividad 7

a) 1000 → 0100 → 0010 → 0001 → 1000 . . . : 4 estados distintos, uno por etapa.

b) Porque el estado 0000 también es estable dentro del anillo: si al encender todos los flip-flops quedan en 0, ese 0 circula indefinidamente y ninguna lámpara se enciende nunca. Con Preset/Clear se fuerza el estado inicial 1000 al energizar.

c) 0000 → 1000 → 1100 → 1110 → 1111 → 0111 → 0011 → 0001 → 0000 . . . : 8 estados ($2N$ con $N = 4$), que es exactamente el efecto de llenado y vaciado pedido.

d) Anillo: 8 etapas, 8 flip-flops, sin decodificación. Johnson: 4 flip-flops, sin decodificación (o con compuertas muy simples si se quiere una salida por estado). Binario: 3 flip-flops, pero exige un decodificador de 3 a 8. El Johnson es el mejor compromiso cuando se busca poca lógica adicional.

Actividad 8

a) 3 flip-flops, porque con 2 solo se llega a 4 estados. El módulo natural de 3 flip-flops es $2^3 = 8$.

b) Hay que detectar el estado 110_2 (6) y usarlo para resetear. Como basta con que Q_2 y Q_1 estén en 1 (no existe otro estado de la secuencia con esa combinación), alcanza con una compuerta AND de dos entradas conectada a Q_2 y Q_1.

c) 000 → 001 → 010 → 011 → 100 → 101 → 000 . . . : 6 estados, módulo 6.

d) Con reset asíncrono el contador entra realmente en el estado 6 y recién ahí se borra: durante ese lapso muy breve la decodificación de fases puede activar una salida equivocada (un destello de la fase 7 en el semáforo, o un pulso espurio en un relé). El reset síncrono fuerza la vuelta a 000 en el mismo flanco de reloj, por lo que el estado 6 nunca llega a existir en las salidas.

Actividad 9

a) Unidades: módulo 10 (0 a 9, contador de décadas). Decenas: módulo 6 (0 a 5).

b) La salida de fin de cuenta de las unidades se activa en la transición de 9 a 0 y habilita el avance de las decenas en ese mismo momento. En cascada síncrona, esa señal se conecta a la entrada de habilitación de conteo del segundo contador, compartiendo ambos el mismo reloj.

c) $10 \times 6 = 60$. El módulo total de contadores en cascada es el producto de los módulos individuales, porque las decenas avanzan una vez por cada ciclo completo de las unidades.

d) Un pulso cada 60 segundos, es decir, una vez por minuto. Esa señal es la que alimenta el contador de minutos del reloj.

e) Hay que dividir por 1000, que se descompone como $10 \times 10 \times 10$: tres contadores de décadas en cascada. La salida de fin de cuenta del tercero entrega 1 Hz.

Actividad 10

a) 3 flip-flops ($2^3 = 8$ estados, uno por piso).

b) Un contador síncrono ascendente/descendente. La señal de dirección selecciona, mediante la lógica de habilitación de cada etapa, si el contador avanza hacia el estado siguiente o hacia el anterior. El pulso del sensor de piso funciona como reloj.

c) Porque el contador es cíclico: al llegar a 111_2 (7) y recibir otro pulso, desborda y vuelve a 000_2 (0). La solución es inhibir el conteo cuando se cumplan simultáneamente "estado $= 7$" y "dirección $=$ subiendo": una compuerta AND de Q_2, Q_1 y Q_0 combinada con la señal de dirección, cuyo resultado bloquea el reloj o la habilitación de conteo. Conviene hacer lo simétrico para el estado 0 bajando.

d) El piso 5 es 101_2, de modo que hay que detectar $Q_2 = 1$, $Q_1 = 0$, $Q_0 = 1$: una compuerta AND de tres entradas conectada a Q_2, $\overline{Q_1}$ y Q_0 (es decir, Q_1 pasa por un inversor, o se toma directamente la salida $\overline{Q_1}$ del flip-flop).

Capítulo 2

Memorias Digitales

2.1. ¿Qué es una memoria?

La memoria es la parte de un sistema digital que almacena datos binarios en grandes cantidades. Está formada por matrices de elementos de almacenamiento —biestables o capacitores— capaces de retener un bit de información cada uno.

En un sistema microprogramable la memoria cumple dos funciones distintas:

- **Memoria de programa:** almacena las instrucciones que la CPU va a ejecutar. No se modifica durante la ejecución del programa. Se implementa en memoria no volátil (Flash, ROM).
- **Memoria de datos:** almacena las variables y resultados intermedios del programa. Se modifica constantemente durante la ejecución. Se implementa en RAM (volátil).

2.2. Unidades de datos binarios

Unidad	Tamaño	Descripción
Bit	1 bit	Unidad mínima de información: un 0 o un 1
Nibble	4 bits	Medio byte; útil en representación BCD
Byte	8 bits	Unidad básica de almacenamiento en la mayoría de los sistemas
Palabra (*word*)	variable	Unidad completa de trabajo de la CPU (8, 16, 32 o 64 bits según la arquitectura)

Las capacidades de memoria se expresan en múltiplos de byte:

Múltiplo	Equivale a
1 Kbyte (KB)	2^{10} bytes $=$ 1.024 bytes
1 Megabyte (MB)	2^{20} bytes $=$ 1.048.576 bytes
1 Gigabyte (GB)	2^{30} bytes $=$ 1.073.741.824 bytes

2.3. Organización interna: la matriz de memoria

Internamente, una memoria se organiza como una **matriz de celdas**, donde cada celda almacena un bit. La forma en que se agrupan esas celdas determina la organización de la memoria.

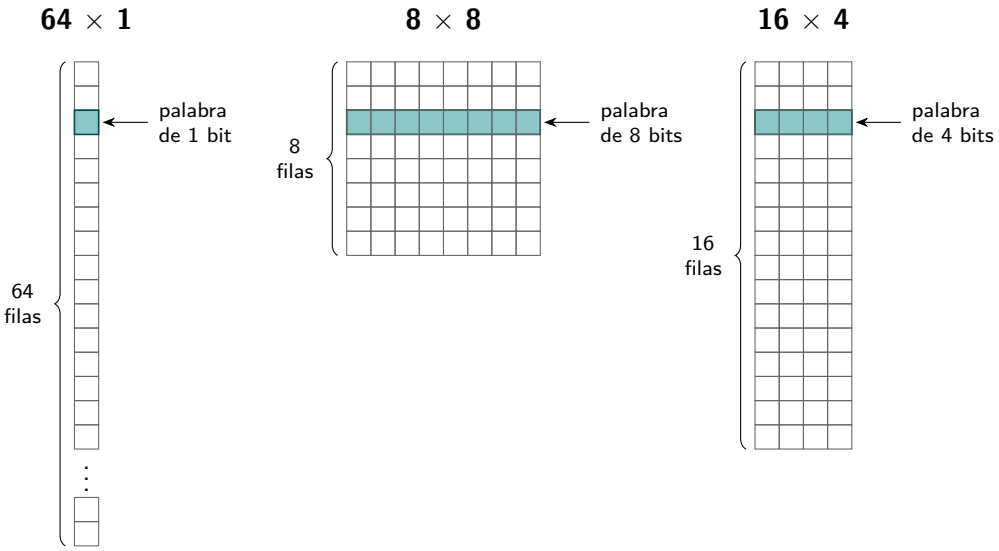


Figura 2.1: Una misma matriz de 64 celdas organizada como 64×1, 8×8 y 16×4. Cada fila es una palabra (resaltada).

Por ejemplo, una matriz de 64 celdas puede organizarse como:

- **64 × 1:** 64 palabras de 1 bit cada una.
- **8 × 8:** 8 palabras de 8 bits (8 bytes).
- **16 × 4:** 16 palabras de 4 bits (16 nibbles).

La **capacidad** de una memoria se calcula como:

$$\text{Capacidad} = N_{\text{direcciones}} \times \text{longitud de palabra}$$

Ejemplos:

- Una memoria 16 × 4: 16 posiciones × 4 bits = 64 bits en total.
- Una memoria 16K × 8: 16.384 posiciones × 8 bits = 131.072 bits = 16 KB.

Nota

Los números de posiciones son siempre potencias de 2. El "16k" es en realidad $2^{14} = 16.384$, redondeado a la unidad más próxima por convención.

2.4. Direccionamiento

Cada posición de la matriz tiene una **dirección**: un código binario único que la identifica. Para acceder a una posición se coloca su dirección en el **bus de direcciones**; un decodificador interno selecciona la fila y columna correspondientes.

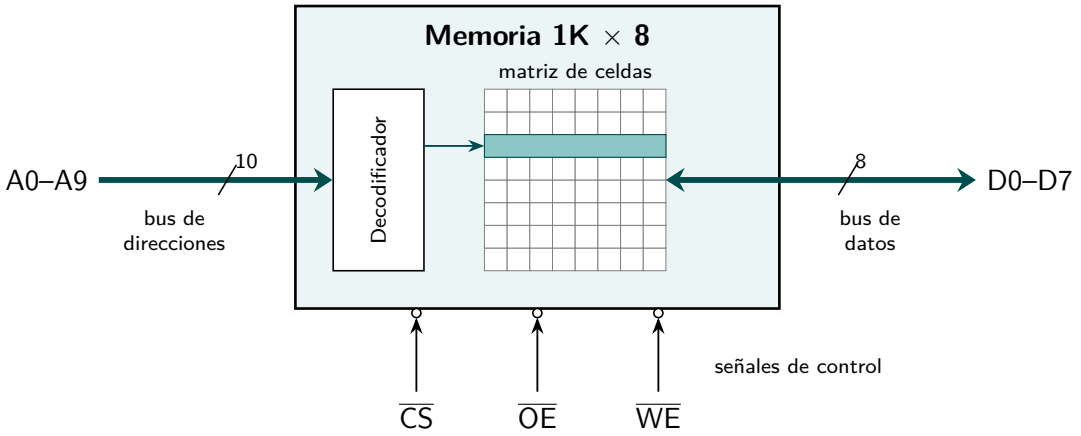


Figura 2.2: Memoria 1K×8: bus de direcciones (A0–A9), bus de datos bidireccional (D0–D7) y señales de control activas en bajo ($\overline{CS}$, $\overline{OE}$, $\overline{WE}$).

La cantidad de posiciones direccionables depende del número de líneas del bus de direcciones:

$$N_{\text{posiciones}} = 2^{N_{\text{líneas de dirección}}}$$

Ejemplo: una memoria con 10 líneas de dirección (A0–A9) puede tener $2^{10} = 1.024$ posiciones.

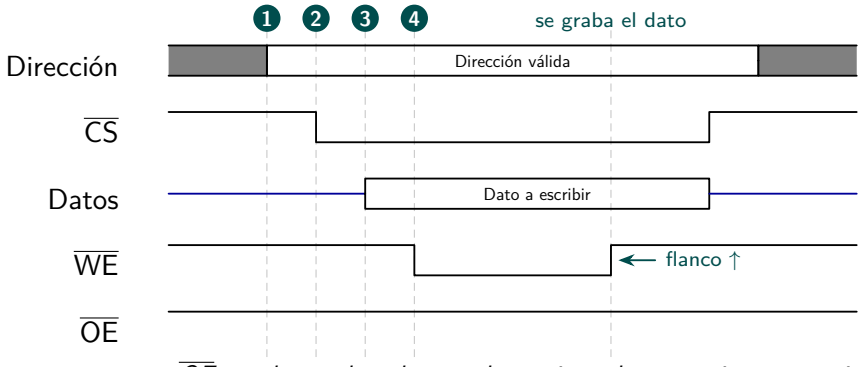
2.5. Operaciones básicas: lectura y escritura

Para controlar si una operación es de lectura o escritura, y para habilitar o deshabilitar el chip, las memorias disponen de señales de control. Las más comunes son:

Señal	Nombre	Función
$\overline{CS}$ (o $\overline{CE}$)	*Chip Select / Chip Enable*	Activa el chip (activa en bajo). Permite conectar varios chips al mismo bus y seleccionar solo uno a la vez
$\overline{OE}$	*Output Enable*	Habilita las salidas de datos durante una lectura
$\overline{WE}$	*Write Enable*	Habilita la escritura de datos

2.5.1. Operación de escritura

1. Se coloca la dirección en el bus de direcciones.
2. Se activa $\overline{CS}$ (se pone en bajo).
3. Se coloca el dato en el bus de datos.
4. Se activa $\overline{WE}$ (pulso en bajo): el dato queda grabado en la posición seleccionada.



$\overline{OE}$ queda en alto: durante la escritura la memoria no maneja el bus de datos.

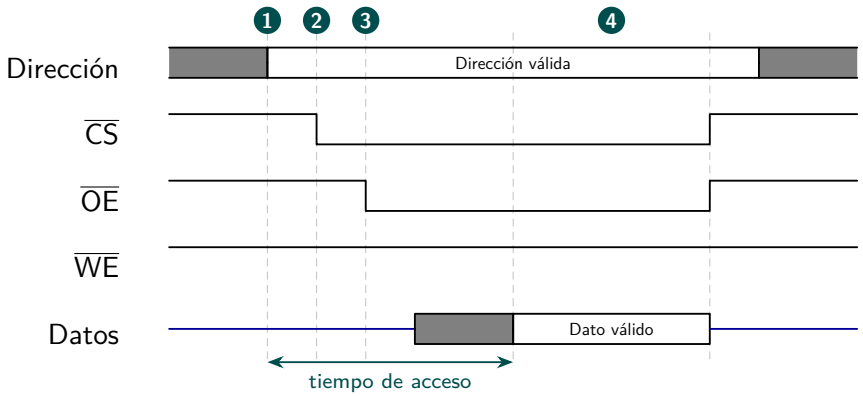
Figura 2.3: Diagrama de tiempos de una escritura. Los números corresponden a los pasos del texto; el dato se graba en el flanco ascendente de $\overline{WE}$.

Atención

Escribir en una posición **destruye** el dato que había antes en esa dirección.

2.5.2. Operación de lectura

1. Se coloca la dirección en el bus de direcciones.
2. Se activa $\overline{CS}$ (se pone en bajo).
3. Se activa $\overline{OE}$ (se pone en bajo): la memoria deposita el dato almacenado en esa dirección sobre el bus de datos.
4. La CPU lee el dato del bus.



$\overline{WE}$ queda en alto: la lectura no modifica el contenido de la memoria.

Figura 2.4: Diagrama de tiempos de una lectura. Los números corresponden a los pasos del texto; el tiempo de acceso va desde que la dirección es válida hasta que el dato es válido.

Nota

La lectura es **no destructiva**: el dato permanece almacenado en la memoria después de ser leído.

2.6. Clasificación de las memorias

Las memorias se clasifican según dos criterios principales:
Según su posibilidad de escritura:

- **ROM** (*Read Only Memory*): solo lectura.
- **RAM** (*Random Access Memory*): lectura y escritura.

Según su volatilidad:

- **Volátiles:** pierden los datos al retirar la alimentación (RAM).
- **No volátiles:** conservan los datos sin alimentación (ROM y sus variantes).

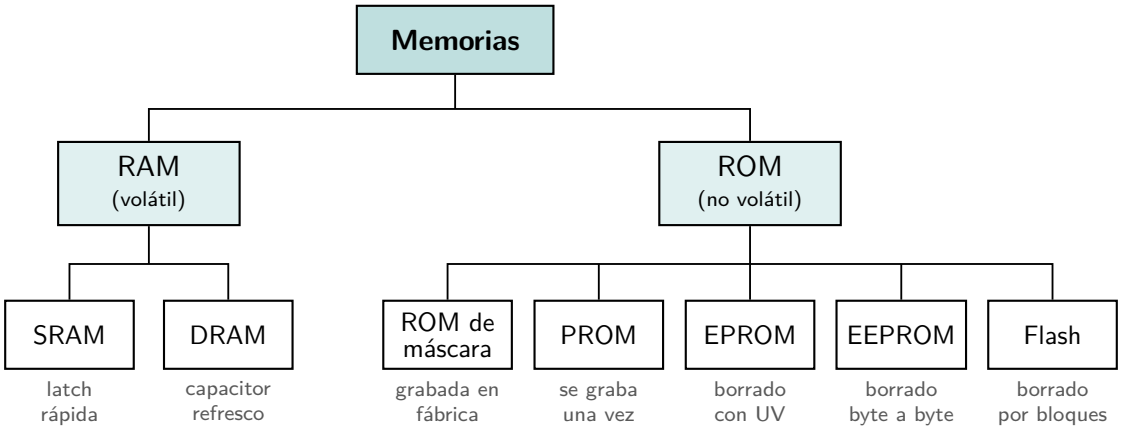


Figura 2.5: Clasificación de las memorias semiconductoras.

2.7. Memorias RAM

2.7.1. RAM estática (SRAM)

Utiliza **biestables (latches o flip-flops)** como elementos de almacenamiento. Cada celda almacena un bit de forma estable mientras haya alimentación, sin necesidad de refresco.

2.7.1.1. La idea: una celda hecha con un latch D

La forma más directa de construir una celda de memoria es con el **latch D con habilitación** que vimos en el Capítulo 1. Alrededor del latch se agregan dos compuertas que deciden cuándo se puede escribir y cuándo se puede leer:

- **Escritura:** el latch se habilita solo si la fila está seleccionada *y* se ordena escribir ($EN = \text{Selección} \cdot \text{Escribir}$). En ese momento copia el bit de *Entrada de datos*. En cualquier otro caso $EN = 0$ y el latch **retiene** el bit.
- **Lectura:** un **buffer tri-estado** conecta la salida Q con *Salida de datos* solo si la fila está seleccionada *y* se ordena leer. Si no, la salida queda en alta impedancia y no interfiere con las otras celdas que comparten la misma línea de datos.

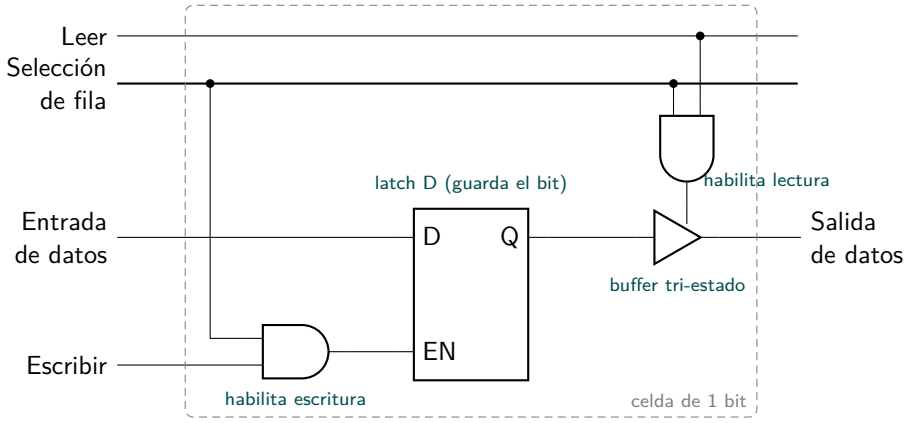


Figura 2.6: Celda SRAM básica construida con un latch D: la selección de fila habilita la escritura (junto con *Escribir*) y la lectura (junto con *Leer*).

2.7.1.2. La optimización: celda de 6 transistores (6T)

La celda anterior funciona, pero es **cara**: cada compuerta lógica está hecha con varios transistores, y la celda completa necesita más de 20. En un chip con millones de celdas eso es inaceptable. Las SRAM reales usan una versión reducida a lo esencial:

- El latch se reduce a **dos inversores acoplados en cruz** (4 transistores): la salida de cada uno alimenta la entrada del otro, y el bit queda "trabado" en los nodos Q y $\overline{Q}$.
- Las compuertas de habilitación y el buffer se reemplazan por **dos transistores de acceso**, que funcionan como llaves.
- La **línea de palabra** (WL, *word line*) cumple el papel de la selección de fila: cuando vale 1 cierra las dos llaves.
- La entrada y la salida de datos se unifican en una misma **línea de bit** (BL, *bit line*), acompañada por su complemento $\overline{\text{BL}}$. Para **escribir**, el circuito externo fuerza BL y $\overline{\text{BL}}$ con el valor deseado y "vence" al latch; para **leer**, se observa cuál de las dos líneas baja.

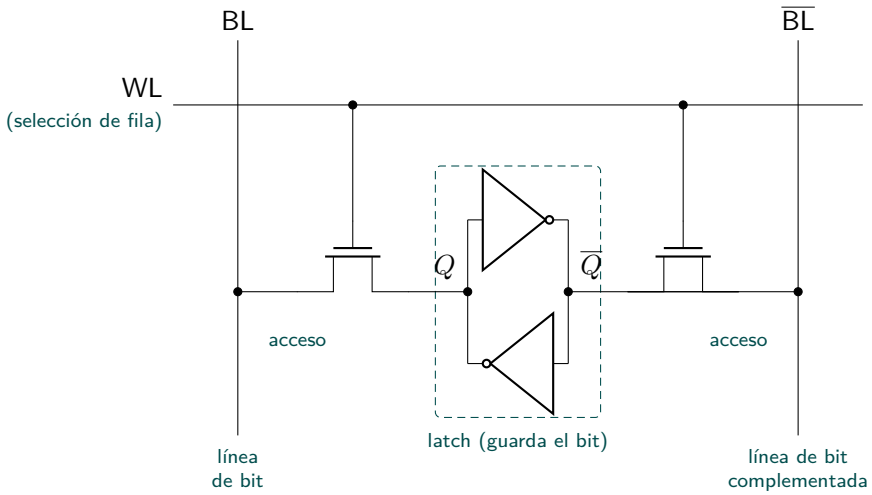


Figura 2.7: Celda SRAM de 6 transistores: latch de dos inversores (4 transistores) y dos transistores de acceso controlados por la línea de palabra (WL), conectados a las líneas de bit BL y $\overline{BL}$.

Nota

Las dos celdas hacen lo mismo: guardan un bit en un biestable y lo conectan al exterior solo cuando la fila está seleccionada. La celda 6T es la que se fabrica realmente porque ocupa mucha menos área de silicio.

Características:

- Mantiene el dato indefinidamente con alimentación.
- Alta velocidad de acceso.
- Mayor área de silicio por bit $\rightarrow$ mayor costo y menor densidad.
- Se usa en cachés de CPU, registros de microcontroladores y en la memoria de datos (SRAM) de los microcontroladores.

2.7.2. RAM dinámica (DRAM)

Utiliza **capacitores** como elementos de almacenamiento. Un transistor controla el acceso al capacitor. La carga del capacitor representa el bit almacenado.

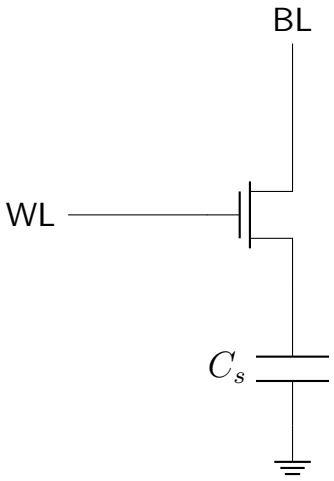


Figura 2.8: Celda básica DRAM: transistor MOS y capacitor de almacenamiento

Características:

- El capacitor se descarga lentamente → necesita un **ciclo de refresco** periódico para no perder el dato.
- Mayor densidad de almacenamiento (celda más pequeña) → menor costo por bit.
- Requiere circuitería adicional para el refresco.
- Es la tecnología utilizada en los módulos de RAM de las computadoras personales (DDR, DDR4, etc.).

2.8. Memorias ROM y sus variantes

2.8.1. ROM de máscara

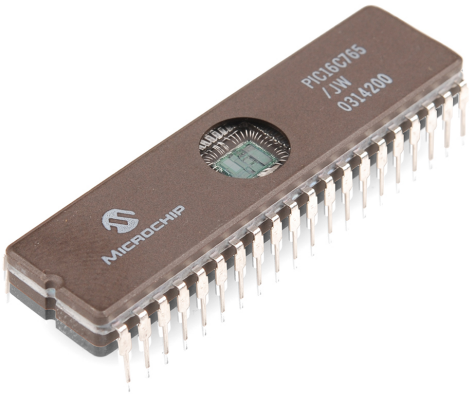
Programada durante la fabricación del chip. El contenido no puede modificarse de ninguna manera. Solo es económica en grandes volúmenes de producción.

2.8.2. PROM (*Programmable ROM*)

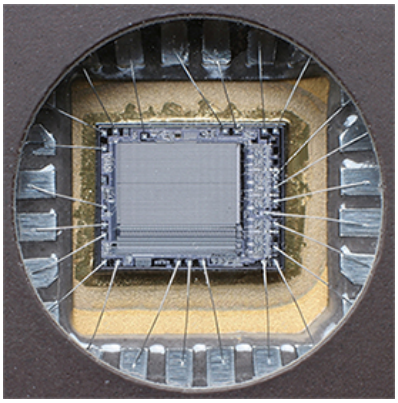
Puede ser programada **una única vez** por el usuario mediante un programador de memoria. El proceso es irreversible: una vez programada no puede cambiarse. También llamada OTP (*One Time Programmable*).

2.8.3. EPROM (*Erasable PROM*)

Puede ser **borrada mediante luz ultravioleta** a través de una ventana de cuarzo en el encapsulado, y luego reprogramada. El borrado es total (no selectivo) y tarda varios minutos. Reconocible por la ventana transparente en el chip.



(a) Microcontrolador PIC16C765/JW: la versión "JW" tiene la memoria de programa en EPROM y se reconoce por la ventana.



(b) Detalle de la ventana de cuarzo: a través de ella se ve el chip de silicio y los hilos de conexión.

Figura 2.9: EPROM con ventana de cuarzo. Para borrarla se expone el chip a luz ultravioleta a través de la ventana; después se tapa con una etiqueta opaca para que la luz ambiente no la borre de a poco.

2.8.4. EEPROM (*Electrically Erasable PROM*)

Puede ser **borrada y reprogramada eléctricamente**, byte a byte, sin necesidad de extraer el chip del circuito. Permite actualizaciones en el propio sistema (*In-System Programming*). Velocidad de escritura más lenta que la RAM; número de ciclos de escritura limitado (típicamente 100.000 a 1.000.000).

2.8.5. Memoria Flash

Es la evolución de la EEPROM. Comparte la capacidad de borrado y escritura eléctrica, pero opera por **bloques** en lugar de byte a byte, lo que la hace significativamente más rápida en operaciones de borrado. Combina:

- Alta densidad de almacenamiento.
- No volatilidad.
- Lectura/escritura en el propio circuito.
- Velocidad de operación comparativamente alta.
- Bajo costo por bit.

Por estas razones es la tecnología dominante en memorias USB, tarjetas SD y en la **memoria de programa de los microcontroladores modernos** (incluyendo el ATmega328, que tiene 32 KB de Flash).

Comparativa de tipos de ROM:

Tipo	Programación	Borrado	Granularidad borrado
ROM de máscara	En fábrica	No borrable	—
PROM	Usuario, 1 vez	No borrable	—
EPROM	Usuario	UV, total	Todo el chip
EEPROM	Eléctricamente	Eléctricamente	Byte a byte
Flash	Eléctricamente	Eléctricamente	Por bloques

2.9. Expansión de memoria

En muchas aplicaciones la capacidad o el ancho de palabra de un único chip de memoria no son suficientes. Se pueden combinar varios chips para ampliar la memoria total del sistema. Existen dos técnicas:

2.9.1. Expansión de palabra (ampliación de bits por posición)

Se usa cuando el ancho de palabra de los chips es menor que el del bus de datos del sistema. Los chips se conectan **en paralelo**, compartiendo el mismo bus de direcciones y las mismas señales de control, pero cada uno aporta algunos bits del bus de datos.

Ejemplo: el bus de datos del sistema es de 8 bits, pero se tienen chips de 4 bits de ancho (organizados como 1K × 4). Se conectan dos chips en paralelo: uno proporciona D0–D3 y el otro D4–D7. Ambos chips responden a la misma dirección simultáneamente.

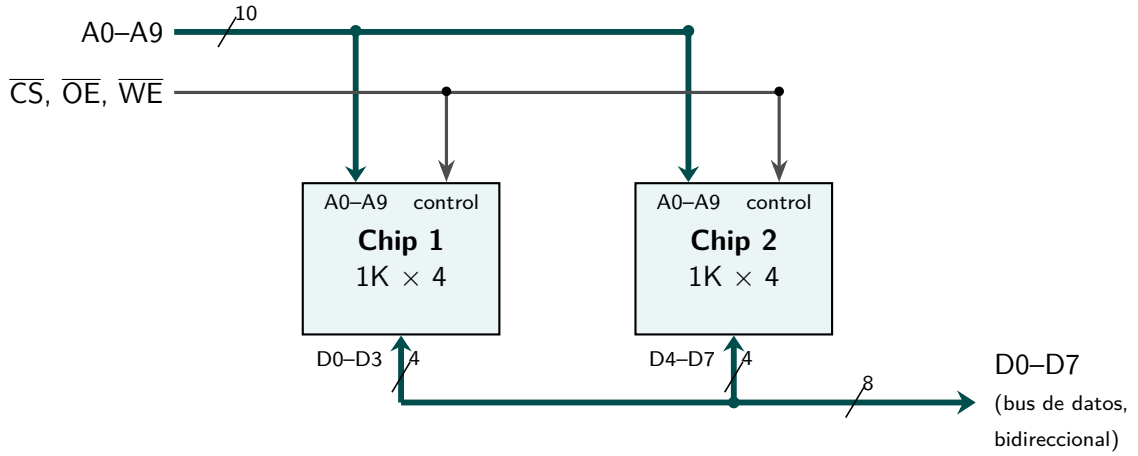


Figura 2.10: Expansión de palabra: dos chips 1K×4 en paralelo forman una memoria 1K×8.

$$\text{Chips necesarios} = \frac{\text{ancho de palabra del sistema}}{\text{ancho de palabra del chip}}$$

2.9.2. Expansión de capacidad (ampliación de direcciones)

Se usa cuando la cantidad de posiciones de un único chip es menor que la requerida. Los chips se conectan de forma que cada uno responde a un rango diferente de direcciones. Se usa la señal $\overline{CS}$ de cada chip para habilitarlo solo cuando el bus de direcciones está en su rango.

Ejemplo: se necesitan 4K posiciones de 8 bits, pero solo se tienen chips de 1K $\times$ 8. Se conectan 4 chips: el primero responde a las direcciones 0x000–0x3FF, el segundo a 0x400–0x7FF, y así sucesivamente. Un decodificador de 2 a 4 (por ejemplo, el 74139, o un 74138 usando 4 de sus 8 salidas) genera las señales $\overline{CS}$ individuales a partir de las líneas de mayor peso del bus de direcciones.

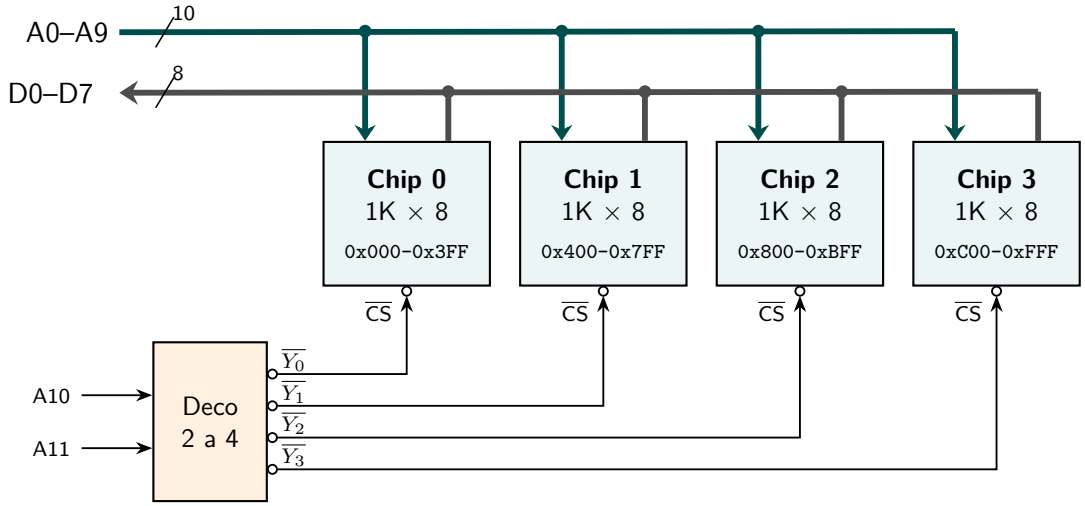


Figura 2.11: Expansión de capacidad: cuatro chips 1K×8 y un decodificador 2 a 4 forman una memoria 4K×8.

$$\text{Chips necesarios} = \frac{\text{capacidad total requerida}}{\text{capacidad de cada chip}}$$

2.9.3. Expansión combinada

Es posible aplicar ambas técnicas simultáneamente para ampliar tanto el ancho de palabra como la cantidad de posiciones. En ese caso:

$$\text{Chips necesarios} = \frac{\text{capacidad total}}{\text{capacidad del chip}} \times \frac{\text{ancho de palabra del sistema}}{\text{ancho de palabra del chip}}$$

Capítulo 3

Sistemas Microprogramables

3.1. Lógica cableada y lógica programada

En electrónica digital se pueden diferenciar dos grandes enfoques para resolver un problema:

Lógica cableada: Para cada aplicación se diseña un circuito específico basado en compuertas lógicas. Una vez construido es muy difícil o imposible de modificar. El comportamiento del sistema queda determinado por el hardware.

Lógica programada: Un mismo dispositivo puede resolver distintos problemas simplemente cambiando su programa. El comportamiento del sistema queda determinado por el software que se le carga.

3.2. Definición de sistema microprogramable

Un sistema microprogramable es un dispositivo, o conjunto de dispositivos, de propósito general que se programa para resolver distintos problemas. Su versatilidad radica en que basta con cambiar el software para cambiar completamente su función.

Algunas aplicaciones típicas:

- **Informática:** computadoras personales, periféricos (teclado, impresora, escáner).
- **Cálculo:** calculadoras científicas, supercomputadoras.
- **Procesos industriales:** PLCs para control de robots y líneas de montaje.
- **Electrodomésticos:** lavadoras, hornos, televisores, reproductores de audio/video.
- **Comunicaciones:** teléfonos fijos y móviles, equipos de red.
- **Automoción:** control de climatización, ABS, airbags.
- **Supervisión y alarmas:** ascensores, calefacción, alarmas contra incendio y robo.
- **Otros:** instrumentación, electromedicina, sistemas de navegación, tarjetas inteligentes (SUBE, monedero electrónico).

3.3. Clasificación de los sistemas microprogramables

Los sistemas microprogramables se clasifican según su grado de integración, capacidad de procesamiento y forma de trabajo:

3.3.1. Microprocesadores

El sistema está distribuido en varios circuitos integrados. El más importante es el **microprocesador**, que contiene únicamente la Unidad Central de Proceso (CPU). La memoria, los puertos de entrada/salida y demás periféricos son componentes externos que se interconectan mediante buses.

Se utiliza cuando se requieren grandes cantidades de memoria, alta velocidad o se prevén futuras ampliaciones. Ejemplo típico: la computadora personal.

3.3.2. Microcontroladores

Todo el sistema microprogramable —CPU, memoria, puertos de E/S y periféricos— está integrado en un **único circuito integrado**. Esto reduce el costo y el tamaño del sistema final, aunque su capacidad de procesamiento es menor que la de los microprocesadores. Son ideales para aplicaciones embebidas donde se necesita un procesador compacto y de bajo consumo.

Se encuentran en prácticamente todos los dispositivos electrónicos modernos: electrodomésticos, juguetes, instrumentos, automóviles, etc.

3.3.3. Dispositivos PLDs

Los PLD (*Programmable Logic Device*) integran una matriz de compuertas lógicas cuya interconexión puede ser programada por el usuario. A diferencia de los microprocesadores y microcontroladores, **no están basados en una CPU** que lee instrucciones de una memoria: su lógica queda grabada en la estructura del dispositivo.

Son adecuados para resolver problemas combinacionales o secuenciales de alta velocidad pero de lógica fija, como decodificadores, máquinas de estado o interfaces de hardware.

3.4. Hardware, Software y Firmware

Un sistema microprogramable se compone de tres elementos:

- **Hardware:** la parte física del sistema; el conjunto de circuitos electrónicos.
- **Software:** el conjunto de programas e instrucciones que se cargan en el sistema para indicarle qué hacer. También se denomina software al conjunto de lenguajes utilizados para escribir esos programas.
- **Firmware:** software grabado en la memoria interna del dispositivo que el usuario normalmente no puede modificar. Cumple la función de intermediario entre el hardware y el software de aplicación. En una PC, la BIOS es un ejemplo de firmware.

3.5. Arquitectura de un sistema basado en CPU

Todo sistema microprogramable (ya sea basado en microprocesador o en microcontrolador) comparte la misma estructura fundamental. Sus elementos esenciales son:

- Reloj
- CPU (ALU + Registros + Unidad de Control)
- Memoria
- Unidad de Entrada/Salida
- Buses del sistema

3.5.1. Reloj

El reloj es un generador de señal cuadrada de frecuencia constante, normalmente producida por un cristal de cuarzo (aunque también pueden usarse osciladores RC). Todos los elementos del sistema operan sincronizados con esta señal, que determina la velocidad de trabajo del sistema.

A partir de la señal de reloj se define el **ciclo de máquina**, compuesto por dos fases:

1. **Fase de búsqueda (Fetch):** la CPU lee la próxima instrucción de la memoria y la almacena en el registro correspondiente.
2. **Fase de ejecución (Execute):** la CPU ejecuta la instrucción leída.

Algunos microprocesadores completan una instrucción en un único ciclo de máquina; otros requieren varios ciclos para instrucciones más complejas. La frecuencia del reloj y la cantidad de ciclos por instrucción determinan juntos la velocidad real de procesamiento.

3.5.2. CPU (Unidad Central de Proceso)

Es el elemento más importante del sistema. Se encarga de:

- Interpretar y ejecutar las instrucciones del programa.
- Generar las señales de control para coordinar el resto del sistema.
- Realizar las operaciones aritméticas y lógicas.
- Transferir datos hacia y desde la memoria.

La CPU está formada por tres bloques:

3.5.2.1. Unidad Aritmético-Lógica (ALU)

Es un circuito combinacional que realiza operaciones aritméticas (suma, resta, multiplicación) y lógicas (AND, OR, NOT, XOR), así como desplazamientos de bits. Las operaciones las realiza sobre datos provenientes de los registros internos o de posiciones de memoria, y deposita el resultado en un registro (habitualmente el acumulador).

3.5.2.2. Registros

Los registros son pequeñas memorias internas de alta velocidad donde la CPU almacena datos e instrucciones de manera temporal. Están formados por biestables y su tamaño (en bits) define la arquitectura del procesador (8, 16, 32 o 64 bits).

Los registros más importantes son:

Registro acumulador: almacena uno de los operandos de la ALU y recibe el resultado de la operación. Todas las operaciones aritméticas y lógicas pasan por él.

Registro de estado (flags): conjunto de biestables que reflejan el resultado de la última operación realizada por la ALU. Los flags más comunes son:

Flag	Nombre	Se activa cuando...
Z	Zero	El resultado de la operación es cero
C	Carry	Hubo acarreo en el bit de mayor peso
N / S	Negativo / Signo	El resultado es negativo
V / OV	Overflow	El resultado excede el rango del tipo de dato
I	Interrupt	Las interrupciones globales están habilitadas

Registros auxiliares o de propósito general: registros que el programador puede usar libremente para almacenar datos intermedios. Su cantidad varía según la arquitectura.

Stack Pointer (SP): puntero que apunta a la cima de la pila (*stack*). La pila es una zona de memoria con estructura LIFO (*Last In, First Out*): el último dato en entrar es el primero en salir. Se utiliza para guardar la dirección de retorno cuando se llama a una subrutina, y para preservar el estado de los registros durante el manejo de interrupciones.

Contador de Programa (PC): registro que contiene la dirección de la próxima instrucción a ejecutar. Se incrementa automáticamente en cada ciclo de fetch. Cuando se produce un salto o llamada a subrutina, este registro se actualiza con la nueva dirección.

3.5.2.3. Unidad de Control (UC)

Gobierna el funcionamiento de todo el sistema. Recibe las instrucciones del programa, las decodifica mediante el **decodificador de instrucciones** y genera las señales de control necesarias para que cada elemento realice su tarea en el momento correcto. También contiene el Contador de Programa.

3.5.3. Memoria

La memoria almacena los programas y los datos que el sistema necesita para funcionar. Se divide en dos zonas funcionales:

- **Memoria de programa:** contiene las instrucciones a ejecutar. No se modifica durante la ejecución. Suele implementarse en memoria no volátil (ROM, Flash).
- **Memoria de datos:** almacena los datos y resultados intermedios del programa. Se modifica constantemente durante la ejecución. Siempre es RAM (volátil).

Nota

No confundir con las memorias de almacenamiento masivo (disco duro, pendrive), que son periféricos externos.

3.5.4. Unidad de Entrada/Salida (E/S)

Permite la comunicación del sistema con el mundo exterior. Está formada por registros que, controlados por los buses del sistema, permiten leer señales del exterior (entradas) o escribir señales hacia afuera (salidas). Su función principal es adaptar las velocidades y niveles lógicos del sistema interno con los de los dispositivos externos.

3.5.5. Buses del sistema

Los elementos del sistema se interconectan mediante **buses**: conjuntos de conductores paralelos que transportan información del mismo tipo. El número de líneas de un bus determina su **ancho** y por ende la cantidad de información que puede transmitirse simultáneamente.

En un sistema basado en CPU existen tres buses fundamentales:

3.5.5.1. Bus de direcciones (*Address Bus*)

Transporta la dirección de la posición de memoria o del registro de E/S que se desea acceder. Es **unidireccional**: solo la CPU pone datos en él.

La cantidad de líneas del bus de direcciones determina la cantidad máxima de posiciones de memoria que el sistema puede direccionar:

$$N_{\text{direcciones}} = 2^{N_{\text{líneas}}}$$

Por ejemplo, un bus de 16 líneas puede direccionar $2^{16} = 65.536$ posiciones. Si cada posición almacena 1 byte, la memoria máxima es 64 KB.

Las líneas se nombran A0 (bit de menor peso) a An (bit de mayor peso).

3.5.5.2. Bus de datos (*Data Bus*)

Transporta los datos entre la CPU, la memoria y los periféricos. Es **bidireccional**: los datos pueden ir en ambas direcciones según se trate de una lectura o escritura. Su ancho define la cantidad de bits que se transfieren en paralelo en cada operación.

Las líneas se nombran D0 a Dn.

3.5.5.3. Bus de control (*Control Bus*)

Transporta las señales de coordinación del sistema: órdenes de lectura/escritura, habilitación de dispositivos, señales de interrupción, reset, etc. Permite que todos los elementos del sistema funcionen en sincronía y sin conflictos en el uso de los buses compartidos.

3.5.5.4. Operaciones de lectura y escritura

Escritura: la CPU coloca la dirección en el bus de direcciones, activa la señal de escritura en el bus de control y coloca el dato en el bus de datos. El dispositivo seleccionado almacena el dato.

Lectura: la CPU coloca la dirección en el bus de direcciones y activa la señal de lectura en el bus de control. El dispositivo seleccionado deposita su contenido en el bus de datos, y la CPU lo lee. La operación de lectura es **no destructiva**: el dato sigue almacenado en la memoria después de ser leído.

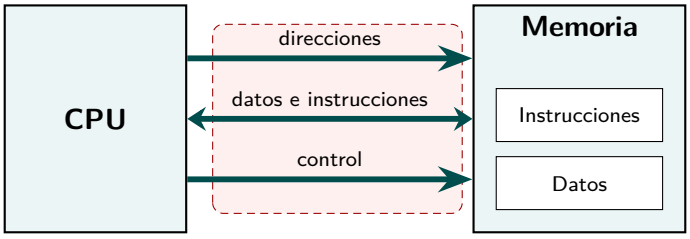
3.6. Arquitecturas Von Neumann y Harvard

La relación entre la CPU y la memoria puede organizarse de dos maneras distintas:

3.6.1. Arquitectura Von Neumann

La CPU se conecta a una **única memoria** mediante un **único juego de buses**. Tanto las instrucciones del programa como los datos se almacenan en el mismo espacio de memoria y viajan por el mismo bus.

► **Diagrama generado en CircuiTikZ — revisar antes de la versión final**



bus único: ***cuello de botella***
(instrucciones y datos se turnan para usarlo)

Figura 3.1: Arquitectura Von Neumann: una única memoria guarda instrucciones y datos, y la CPU accede a ella por un único juego de buses. Ese bus compartido es el cuello de botella.

Limitaciones:

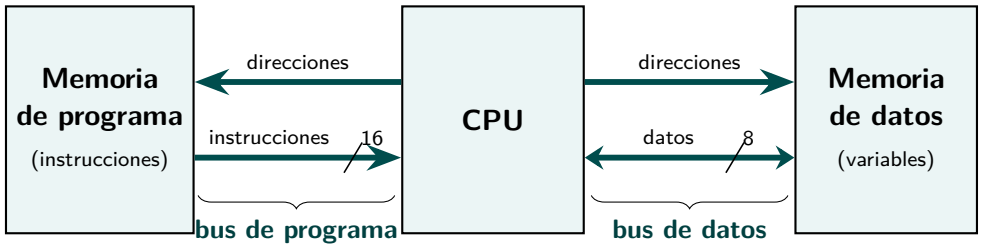
- El bus único genera un cuello de botella: mientras la CPU busca una instrucción no puede acceder a datos, y viceversa.
- El tamaño de las instrucciones está condicionado por el ancho del bus de datos.

Es la arquitectura típica de los microprocesadores de uso general (PCs).

3.6.2. Arquitectura Harvard

La CPU se conecta a **dos memorias separadas** mediante **dos juegos de buses independientes**: uno para instrucciones y otro para datos.

► Diagrama generado en CircuiTikZ — revisar antes de la versión final



Ej. ATmega328P: Flash de programa (instrucciones de 16 bits) / SRAM de datos (8 bits)

Figura 3.2: Arquitectura Harvard: memorias de programa y de datos separadas, cada una con su propio bus, lo que permite buscar una instrucción y acceder a un dato al mismo tiempo.

Ventajas:

- La CPU puede leer la próxima instrucción y acceder a datos **simultáneamente**, eliminando el cuello de botella.
- El ancho del bus de programa puede ser diferente al del bus de datos, permitiendo optimizar ambos de forma independiente.
- Aumenta significativamente la velocidad de procesamiento.

Es la arquitectura utilizada por la mayoría de los **microcontroladores**, incluyendo la familia AVR (ATmega).

3.7. Software: instrucciones y lenguajes de programación

3.7.1. Formato de una instrucción

Las instrucciones que ejecuta la CPU tienen dos partes:

```
+----------------------+----------------------+
|   Codigo de          |      Operando        |
|   operacion          |   (puede no estar)   |
+----------------------+----------------------+
```

- **Código de operación:** indica qué hacer (sumar, restar, mover un dato, comparar, saltar...).
- **Operando:** indica con qué dato realizar la operación, o la dirección de memoria donde se encuentra ese dato.

3.7.2. Tipos de instrucciones

- **Aritméticas:** suma, resta, multiplicación, división.
- **Lógicas:** AND, OR, NOT, XOR.
- **Desplazamiento:** rotación y corrimiento de bits.
- **Transferencia de datos:** mover datos entre registros, entre registros y memoria, o entre registros y puertos E/S.
- **Salto:** condicionales (ejecutan un salto solo si se cumple una condición de los flags) e incondicionales (saltan siempre).
- **Control:** inicialización, detención, no-operación (NOP), habilitación/deshabilitación de interrupciones.

3.7.3. Lenguajes de programación

Para que un programador pueda escribir el programa, y para que luego ese programa pueda ser ejecutado por la CPU, existen distintos niveles de lenguaje:

3.7.3.1. Lenguaje máquina (código máquina)

Es el único lenguaje que la CPU entiende directamente: secuencias de bits (ceros y unos) que representan instrucciones y datos. Aunque es el más eficiente, es extremadamente difícil de escribir y leer por un ser humano.

3.7.3.2. Lenguaje ensamblador (*Assembler*)

Reemplaza los códigos binarios por **mnemónicos**: abreviaturas en inglés que recuerdan la función de cada instrucción (`MOV`, `ADD`, `JMP`, etc.). Las direcciones de memoria pueden referenciarse con etiquetas simbólicas. Es específico de cada arquitectura.

Un programa llamado **ensamblador** traduce el código escrito en lenguaje ensamblador al código máquina correspondiente.

3.7.3.3. Lenguajes de alto nivel

Son los más cercanos al lenguaje humano y los más fáciles de escribir y mantener. No están atados a una arquitectura específica, lo que permite reutilizar el código en distintos sistemas. Algunos ejemplos: C, C++, Python, Java, Basic.

La CPU no entiende estos lenguajes: solo ejecuta código máquina. Por lo tanto, el programa escrito por el programador (el **código fuente**) siempre tiene que ser traducido. Según **cuándo** se hace esa traducción, los lenguajes se dividen en dos grandes grupos: **compilados** e **interpretados**.

Lenguajes compilados. Un programa llamado **compilador** lee *todo* el código fuente y lo traduce a código máquina *una sola vez*, antes de ejecutarlo. El resultado es un archivo ejecutable (en una PC, un `.exe`; en un microcontrolador, el `.hex` que se graba en la memoria Flash). A partir de ahí, la CPU ejecuta directamente ese código máquina y el compilador ya no participa. Ejemplos: C, C++, Rust.

Lenguajes interpretados. No se genera un ejecutable. Un programa llamado **intérprete**, que está corriendo en la CPU, lee el código fuente *mientras* lo ejecuta: toma una instrucción, la analiza, la ejecuta, pasa a la siguiente, y así hasta el final. Cada vez que se corre el programa, la traducción se vuelve a hacer. Ejemplos: Python, JavaScript, y también PSeInt, que usamos en el Bloque 4.

Nota

Una analogía: compilar es como encargar la traducción completa de un libro: lleva tiempo, pero después cualquiera lo lee de corrido en su idioma. Interpretar es como tener un intérprete simultáneo al lado que va traduciendo frase por frase: se puede empezar enseguida, pero cada vez que se quiera "leer" de nuevo hace falta el intérprete, y todo va más lento.

► **Diagrama generado en CircuiTikZ — revisar antes de la versión final**

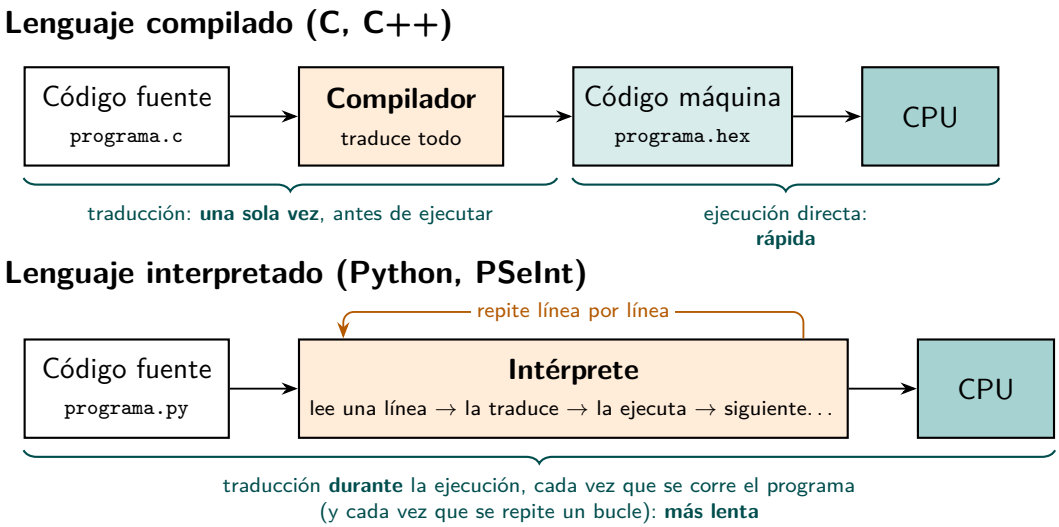


Figura 3.3: Lenguaje compilado: la traducción se hace una vez y la CPU ejecuta el código máquina resultante. Lenguaje interpretado: el intérprete traduce y ejecuta línea por línea cada vez que se corre el programa.

¿Por qué un programa interpretado es más lento? Porque el trabajo de traducir se repite en cada ejecución. Pensemos en un bucle que se repite un millón de veces: en un lenguaje compilado, las instrucciones del bucle se tradujeron una sola vez y la CPU solo las ejecuta; en uno interpretado, el intérprete tiene que leer y analizar esas mismas instrucciones un millón de veces. Además, el intérprete es un programa más que compite por el tiempo de CPU y ocupa memoria. Como resultado, un programa interpretado suele ser **decenas de veces más lento** que el mismo programa compilado.

	Compilado	Interpretado
¿Cuándo se traduce?	Una vez, antes de ejecutar	Durante cada ejecución, línea por línea
¿Qué se ejecuta?	El código máquina generado	El intérprete, que lee el código fuente
Velocidad	Alta	Baja (decenas de veces más lento)
Memoria necesaria	Solo el programa	El programa *y* el intérprete
Detección de errores	Antes de ejecutar (el compilador avisa)	Recién cuando se llega a la línea con error
Probar un cambio	Hay que volver a compilar	Se ejecuta directamente
Portabilidad	Hay que compilar para cada arquitectura	Corre donde exista el intérprete
Ejemplos	C, C++, Rust	Python, JavaScript, PSeInt

Importante

¿Por qué C en un microcontrolador? El ATmega328 tiene solo 32 KB de Flash y 2 KB de RAM, y trabaja a 16 MHz. Ahí no entra un intérprete de Python (necesita cientos de KB), y aunque entrara, el programa sería demasiado lento para controlar hardware en tiempo real. Por eso en sistemas embebidos el lenguaje más utilizado es **C**: es compilado, genera código máquina compacto y rápido, y permite acceder directamente a los registros del hardware, sin dejar de ser legible.

Nota

En la práctica existen también caminos intermedios. Por ejemplo, Java y Python traducen primero el código fuente a un código intermedio (*bytecode*) que luego interpreta una "máquina virtual". Pero la idea central es la misma: cuanto más trabajo de traducción se hace antes de ejecutar, más rápido corre el programa.

3.8. El ATmega328 en este curso

El microcontrolador utilizado en este curso es el **ATmega328P**, fabricado por Microchip (familia AVR). Se trabaja sobre placas Arduino Nano como plataforma de desarrollo, pero **sin utilizar el framework ni las bibliotecas de Arduino**: se programa directamente en C, accediendo a los registros del hardware.

Características principales:

Característica	Valor
Arquitectura	Harvard, RISC de 8 bits
Frecuencia máxima	20 MHz
Memoria de programa	32 KB Flash
Memoria de datos (RAM)	2 KB SRAM
Memoria no volátil de datos	1 KB EEPROM
Puertos de E/S digitales	Hasta 23 pines
Periféricos integrados	Timers, UART, SPI, I^2C, ADC (10 bits), Watchdog

Al ser una arquitectura Harvard, el ATmega328 tiene buses separados para memoria de programa (Flash) y memoria de datos (SRAM), lo que le permite ejecutar la mayoría de las instrucciones en un solo ciclo de reloj.

En los bloques siguientes se verá en detalle cómo programar este microcontrolador: cómo se organizan sus registros internos, cómo se configuran sus puertos y cómo se utilizan sus periféricos.

Capítulo 4

Pensamiento Computacional y Algoritmos

4.1. ¿Qué es un algoritmo?

Un **algoritmo** es una secuencia finita de pasos ordenados y precisos que describe cómo resolver un problema. Para que una secuencia de pasos sea considerada un algoritmo debe cumplir las siguientes características:

- **Finito:** debe terminar en algún momento, no puede ejecutarse indefinidamente.
- **Preciso:** cada paso debe ser claro e inequívoco, sin ambigüedades.
- **Definido:** si se ejecuta dos veces con los mismos datos de entrada, siempre produce el mismo resultado.
- **Con entrada y salida:** puede recibir datos del exterior (entradas) y debe producir al menos un resultado (salida).

Nota

Los algoritmos son independientes del lenguaje de programación. Se pueden expresar en lenguaje natural, en pseudocódigo o en un diagrama de flujo, y luego traducirse a cualquier lenguaje de programación.

Ejemplo cotidiano — receta de cocina:

1. Calentar agua hasta que hierva.
2. Agregar los fideos.
3. Esperar 8 minutos.
4. Colar y servir.

Este es un algoritmo: tiene pasos ordenados, es finito y produce un resultado determinado.

4.2. Representación de algoritmos: el diagrama de flujo

Un **diagrama de flujo** es una representación gráfica de un algoritmo. Utiliza símbolos estandarizados conectados por flechas que indican el orden de ejecución de los pasos.

4.2.1. Símbolos básicos

► Diagrama generado en CircuiTikZ — revisar antes de la versión final

Símbolo	Nombre / forma	Uso
Inicio	**Terminal** óvalo / rect. redondeado	Inicio y fin del algoritmo
x ← a + b	**Proceso** rectángulo	Operación, cálculo o asignación de valor
¿x > 0?	**Decisión** rombo	Condición o pregunta con dos salidas (Sí / No)
Leer x	**Entrada/Salida** paralelogramo	Leer datos del usuario o mostrar resultados
A	**Conector** círculo pequeño	Unir partes del diagrama que no caben en una página
⟶	**Flecha** línea con flecha	Indica el orden (flujo) de ejecución entre pasos

Figura 4.1: Símbolos básicos del diagrama de flujo: forma de cada uno y para qué se usa.

4.2.2. Reglas generales

- Todo diagrama empieza con un símbolo **Inicio** y termina con uno o más símbolos **Fin**.
- Las flechas indican la dirección del flujo: normalmente de arriba hacia abajo y de izquierda a derecha.
- Un rombo de decisión siempre tiene exactamente **dos salidas**: una para cuando la condición es verdadera (Sí) y otra para cuando es falsa (No).
- No deben existir caminos que queden sin salida ni flechas que apunten "al vacío".

4.3. Estructuras de control

Todo algoritmo se construye combinando tres estructuras fundamentales:

4.3.1. Secuencia

Los pasos se ejecutan uno tras otro, en el orden en que aparecen. Es la estructura más simple.

▶ **Diagrama generado en CircuiTikZ — revisar antes de la versión final**

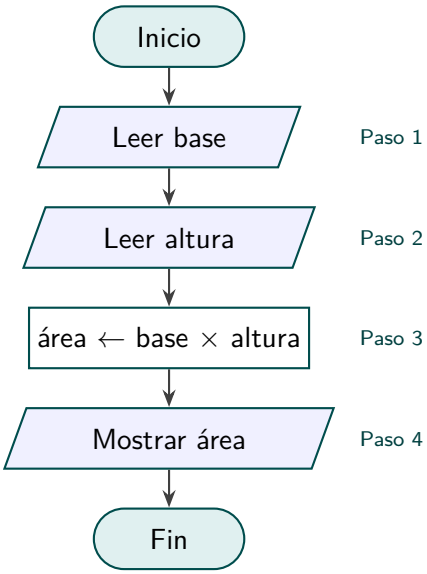


Figura 4.2: Estructura de secuencia: cálculo del área de un rectángulo.

Ejemplo: calcular el área de un rectángulo.

1. Leer base.
2. Leer altura.
3. Calcular área = base × altura.
4. Mostrar área.

4.3.2. Decisión (selección)

El flujo se divide en dos caminos según el resultado de una condición. Se evalúa una expresión lógica (verdadera o falsa) y se ejecuta uno u otro bloque de pasos.

▶ **Diagrama generado en CircuiTikZ — revisar antes de la versión final**

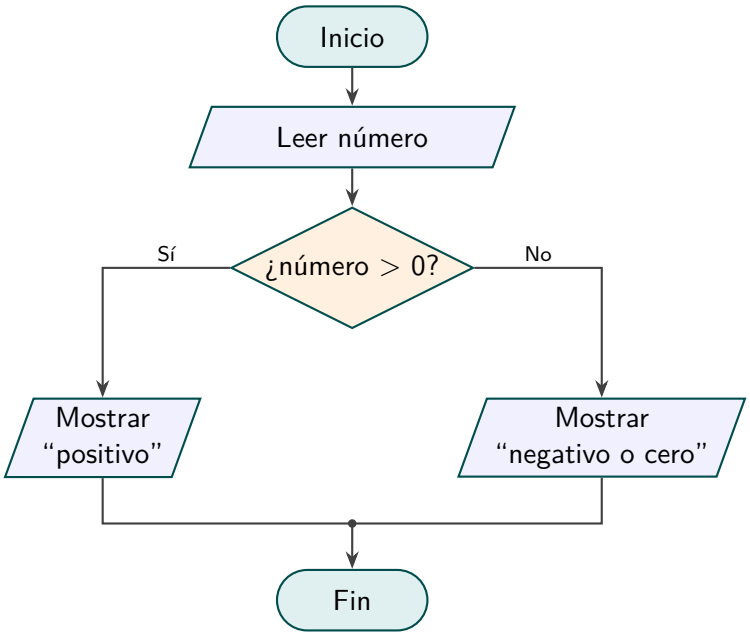


Figura 4.3: Estructura de decisión (si–sino): las ramas Sí y No se reúnen al final.

Ejemplo: determinar si un número es positivo o negativo.

```
Si número > 0 entonces
    Mostrar "El número es positivo"
Sino
    Mostrar "El número es negativo o cero"
Fin Si
```

Las decisiones pueden encadenarse o anidarse para evaluar múltiples condiciones.

4.3.3. Repetición (iteración)

Un bloque de pasos se ejecuta más de una vez, mientras se cumpla una condición. Hay dos variantes principales:

4.3.3.1. Repetir mientras (while — condición al inicio)

La condición se evalúa **antes** de ejecutar el bloque. Si la condición es falsa desde el principio, el bloque no se ejecuta nunca.

▶ **Diagrama generado en CircuiTikZ — revisar antes de la versión final**

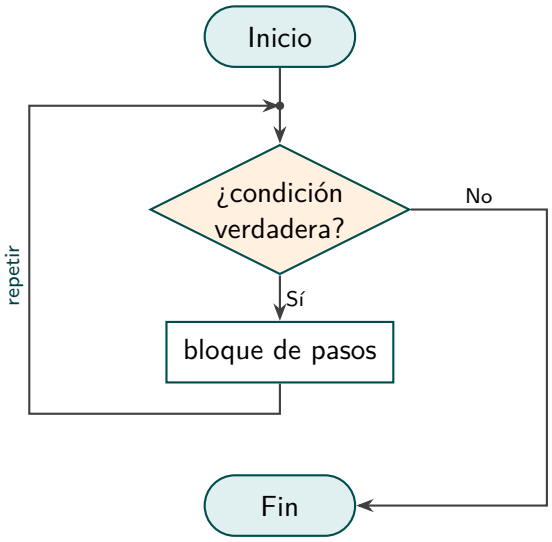


Figura 4.4: Estructura de repetición *mientras*: la condición se evalúa antes del bloque.

```
Mientras condición sea verdadera hacer
    [bloque de pasos]
Fin Mientras
```

Ejemplo: mostrar los números del 1 al 10.

```
contador = 1
Mientras contador <= 10 hacer
    Mostrar contador
    contador = contador + 1
Fin Mientras
```

4.3.3.2. Repetir hasta (do-while — condición al final)

La condición se evalúa **después** de ejecutar el bloque, por lo que el bloque se ejecuta **al menos una vez**. En PSeInt, el bucle se repite *mientras la condición sea falsa* y termina *cuando la condición se vuelve verdadera*: se repite "hasta que" se cumpla.

▶ Diagrama generado en CircuiTikZ — revisar antes de la versión final

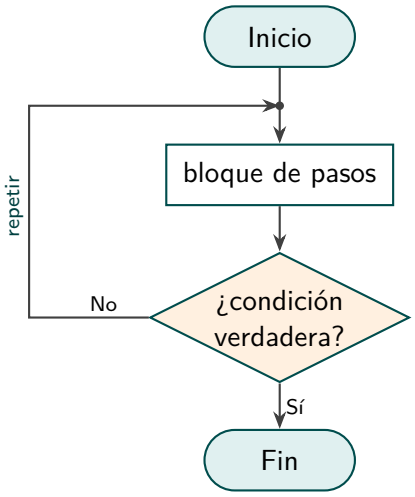


Figura 4.5: Estructura de repetición *repetir–hasta que*: primero se ejecuta el bloque y después se evalúa la condición.

```
Repetir
    [bloque de pasos]
Hasta Que condición
```

Ejemplo: pedir un número hasta que sea positivo.

```
Repetir
    Leer numero
Hasta Que numero > 0
```

Nota

Mientras vs. repetir–hasta: comparando los dos diagramas se ve la diferencia. En el *mientras* el rombo está **antes** del bloque: si la condición es falsa de entrada, el bloque no se ejecuta nunca. En el *repetir–hasta* el rombo está **después**: el bloque siempre se ejecuta al menos una vez. Además, el *mientras* sigue repitiendo con **Sí**, y el *repetir–hasta* sigue repitiendo con **No**.

4.4. Variables

Una **variable** es un espacio de memoria con nombre que almacena un valor que puede cambiar durante la ejecución del algoritmo.

Características:

- Tiene un **nombre** (identificador) que la distingue de las demás.
- Almacena un **tipo de dato** (número entero, número decimal, texto, etc.).
- Su valor puede **leerse** (consultar qué tiene) o **escribirse** (asignarle un nuevo valor).

4.4.1. Tipos de datos básicos

Tipo	Descripción	Ejemplo
Entero	Número sin decimales	5, -3, 100
Real	Número con decimales	3.14, -0.5
Carácter	Un símbolo de texto	'A', '7'
Cadena	Texto (varios caracteres)	"Hola", "temperatura"
Lógico	Verdadero o falso	Verdadero, Falso

4.4.2. Asignación

La asignación guarda un valor en una variable. En pseudocódigo se escribe con la flecha ←:

```
temperatura <- 25
resultado <- base * altura
nombre <- "Juan"
```

4.5. Operadores

4.5.1. Aritméticos

Operador	Operación
+	Suma
-	Resta
*	Multiplicación
/	División
MOD	Módulo (resto de la división entera)

4.5.2. Relacionales (comparación)

Operador	Significado
>	Mayor que
<	Menor que
>=	Mayor o igual que
<=	Menor o igual que
=	Igual
<>	Distinto

4.5.3. Lógicos

Operador	Significado
`Y` / `AND`	Verdadero solo si ambas condiciones son verdaderas
`O` / `OR`	Verdadero si al menos una condición es verdadera
`NO` / `NOT`	Invierte el valor lógico

4.6. Herramienta: PSeInt

PSeInt es un programa que permite escribir y ejecutar algoritmos en pseudocódigo, y visualizarlos como diagrama de flujo de forma automática.

4.6.1. Configuración del perfil

Para este curso se usa el perfil **EEST1 VL**, disponible en el aula virtual (Moodle). Para cargarlo:

1. Ir a **Configurar** → **Opciones del lenguaje (perfiles)** → **Cargar**.
2. Seleccionar el archivo `EEST1_VL`.
3. Hacer clic en **Aceptar**.

4.6.2. Editar el diagrama de flujo

Existen tres formas de acceder al modo de diagrama de flujo:

- Hacer clic en el ícono de diagrama de flujo en la barra de menú.
- Ir a **Archivo** → **Editar diagrama de flujo**.
- Presionar **F7**.

4.6.3. Ejecutar el algoritmo

- **Ejecución completa:** botón *Ejecutar* (flecha verde), o **F9**.
- **Ejecución paso a paso:** botón *Paso a paso* (ícono de pies), útil para seguir el flujo y detectar errores.

4.7. Ejercicios

Los siguientes ejercicios deben resolverse primero dibujando el diagrama de flujo en PSeInt y verificando su funcionamiento con distintos valores de entrada.

4.7.1. Ejercicio 1 — Suma de dos valores

Realizar un algoritmo capaz de sumar 2 valores e informar el resultado.

4.7.2. Ejercicio 2 — Operación con tres valores

Realizar un algoritmo que permita ingresar 3 valores, sumar los primeros 2, restarle el tercero al resultado, e informar el resultado final.

Relación esperada: resultado $= (A + B) - C$

4.7.3. Ejercicio 3 — Módulo de un valor

Realizar un algoritmo capaz de calcular el módulo (valor absoluto) de un número e informar el resultado. Recordar que el módulo devuelve el valor positivo: si el número es positivo lo devuelve igual, si es negativo lo invierte.

Atención

Este ejercicio introduce la estructura de decisión: se necesita evaluar si el número es negativo para invertirlo.

4.7.4. Ejercicio 4 — Sensor de temperatura

Realizar un algoritmo capaz de informar si hace calor o frío. Se ingresa un valor de temperatura y el algoritmo debe:

- Si la temperatura supera los 26°C: informar "hace calor".
- Si la temperatura es menor a 22°C: informar "hace frío".
- En cualquier otro caso (entre 22° y 26°): no informar nada (temperatura agradable).

Atención

Este ejercicio introduce decisiones múltiples: más de una condición debe evaluarse.

4.7.5. Ejercicio 5 — Promedio de una serie de valores

Realizar un algoritmo capaz de calcular el promedio de todos los números ingresados. El ingreso termina cuando el usuario introduce un 0.

Atención

Este ejercicio introduce la estructura de repetición. El algoritmo no sabe de antemano cuántos números se van a ingresar: debe seguir pidiendo valores hasta que se ingrese un 0, y recién entonces calcular y mostrar el promedio.

Consideraciones:

- Llevar la cuenta de cuántos valores se ingresaron (contador).

- Llevar la suma acumulada de los valores.
- No incluir el 0 en el cálculo del promedio.
- Tener en cuenta el caso en que se ingrese 0 como primer valor (no hay valores para promediar).

Capítulo 5

Introducción al Lenguaje C

5.1. Del diagrama de flujo al código

En el bloque anterior aprendimos a representar algoritmos con diagramas de flujo. Ahora vamos a traducir esos diagramas a un lenguaje que la computadora puede compilar y ejecutar: el **lenguaje C**.

C es un lenguaje de propósito general creado a principios de los años 70 por **Dennis Ritchie**, en los Laboratorios Bell (EE. UU.). Lo desarrolló para reescribir el sistema operativo **Unix**, que hasta entonces estaba escrito en ensamblador y solo funcionaba en un modelo de computadora: buscaba un lenguaje casi tan eficiente como el ensamblador, pero que permitiera llevar el mismo programa a máquinas distintas. Esa combinación es la que lo sigue haciendo útil hoy. Es el lenguaje más usado en sistemas embebidos y microcontroladores por dos razones principales: es **eficiente** (genera código compacto y rápido) y ofrece **acceso directo al hardware** (registros, puertos, memoria). Todo lo que aprendamos en este bloque se va a aplicar directamente en la programación del microcontrolador.

5.2. Estructura de un programa en C

Todo programa en C tiene la siguiente estructura mínima:

```c
#include <stdio.h>       // Incluir biblioteca estándar de entrada/salida

int main(void) {         // Función principal -- el programa empieza aquí
    // instrucciones
    return 0;            // Indicar que el programa terminó correctamente
}
```

Elementos obligatorios:

- `#include <stdio.h>`: incluye la biblioteca que permite usar `printf` y `scanf` (entrada y salida por consola). En programas para microcontrolador esto cambiará, pero por ahora la usamos para probar en la PC.
- `int main(void)`: función principal. Todo programa en C debe tener exactamente una función `main`. La ejecución empieza en la primera línea de `main` y termina cuando se llega al `return`.
- `{ }`: las llaves definen un **bloque** de código. Todo lo que está entre `{` y `}` pertenece a ese bloque.

- ;: cada instrucción termina con punto y coma. Olvidarlo es uno de los errores más frecuentes.
- `// comentario`: el compilador ignora todo lo que está a la derecha de `//` en esa línea. Los comentarios sirven para explicar el código.

5.3. Variables y tipos de datos

Antes de usar una variable en C hay que **declararla**: indicar su nombre y el tipo de dato que va a almacenar.

```
tipo nombre;
tipo nombre = valor_inicial;
```

5.3.1. Tipos de datos básicos

Tipo	Tamaño típico	Rango aproximado	Uso
`int`	2 o 4 bytes	-32.768 a 32.767 (2 bytes)	Números enteros
`unsigned int`	2 o 4 bytes	0 a 65.535 (2 bytes)	Enteros sin signo
`float`	4 bytes	$\pm 3{,}4 \times 10^{38}$	Números decimales (simple precisión)
`double`	8 bytes	$\pm 1{,}7 \times 10^{308}$	Números decimales (doble precisión)
`char`	1 byte	-128 a 127	Un carácter ASCII o un entero de 8 bits
`unsigned char`	1 byte	0 a 255	Byte sin signo — muy usado en embebidos

Nota

En programación de microcontroladores AVR, el tipo más usado es `uint8_t` (equivalente a `unsigned char`, siempre 8 bits). Por ahora trabajamos con `int` y `float` para mayor comodidad.

5.3.2. Ejemplos de declaración

```
int edad;                // declara una variable entera sin valor
    inicial
int contador = 0;        // declara e inicializa en 0
float temperatura = 25.5;
char letra = 'A';
unsigned int suma = 0;
```

5.3.3. Reglas para nombrar variables

- Solo letras, números y guion bajo _. No puede empezar con un número.
- C distingue mayúsculas de minúsculas: `Temperatura` y `temperatura` son variables distintas.
- No se pueden usar palabras reservadas del lenguaje (`int`, `if`, `while`, `return`, etc.).
- Elegir nombres descriptivos: `temperatura` es mejor que `t` o `x`.

5.4. Operadores

5.4.1. Aritméticos

Operador	Operación	Ejemplo	Resultado
+	Suma	`3 + 2`	5
-	Resta	`5 - 3`	2
*	Multiplicación	`4 * 3`	12
/	División	`7 / 2`	3 (entera si ambos son `int`)
%	Módulo (resto)	`7 % 2`	1

Atención

Atención con la división entera: en C, dividir dos `int` descarta los decimales. `7 / 2` da 3, no `3.5`. Para obtener el decimal hay que usar `float`: `7.0 / 2` da `3.5`.

5.4.2. Relacionales

Operador	Significado
==	Igual (doble signo — no confundir con = de asignación)
!=	Distinto
>	Mayor que
<	Menor que
>=	Mayor o igual
<=	Menor o igual

5.4.3. Lógicos

Operador	Significado
&&	AND lógico
||	OR lógico
!	NOT lógico

5.4.4. Asignación

```c
x = 5;            // asigna 5 a x
x = x + 1;        // incrementa x en 1
x += 1;           // equivalente al anterior (forma abreviada)
x++;              // también equivalente -- incremento en 1
x--;              // decremento en 1
```

5.5. Entrada y salida por consola

Para los ejercicios en PC usamos dos funciones de `<stdio.h>`:

5.5.1. `printf` — mostrar datos en pantalla

```c
printf("Hola mundo\n");              // imprime texto
printf("El resultado es %d\n", x); // imprime texto + valor de x (
    entero)
printf("Temperatura: %.2f\n", t);  // imprime t con 2 decimales
```

Los especificadores de formato más comunes:

Especificador	Tipo
%d	Entero (`int`)
%f	Decimal (`float` o `double`)
%.2f	Decimal con 2 cifras decimales
%c	Carácter (`char`)
%s	Cadena de texto

El \n al final provoca un salto de línea.

5.5.2. `scanf` — leer datos del teclado

```c
int x;
scanf("%d", &x);     // lee un entero y lo guarda en x

float t;
scanf("%f", &t);     // lee un decimal y lo guarda en t
```

Nota

El `&` antes del nombre de la variable es obligatorio en `scanf`. Indica la **dirección de memoria** donde se debe guardar el dato. Olvidarlo causa comportamiento impredecible.

5.6. Estructuras de control

5.6.1. Secuencia

Las instrucciones se ejecutan en orden, de arriba a abajo. No requiere ninguna sintaxis especial.

```c
int a, b, resultado;
scanf("%d", &a);
scanf("%d", &b);
resultado = a + b;
printf("Resultado: %d\n", resultado);
```

5.6.2. Decisión: `if` / `else`

```c
if (condición) {
    // se ejecuta si la condición es verdadera
} else {
    // se ejecuta si la condición es falsa (opcional)
}
```

Para condiciones encadenadas:

```c
if (temperatura > 26) {
    printf("Hace calor\n");
} else if (temperatura < 22) {
    printf("Hace frio\n");
} else {
    printf("Temperatura agradable\n");
}
```

Atención

Errores frecuentes:

- Usar `=` en lugar de `==` dentro del `if`: `if (x = 5)` asigna 5 a `x` y siempre es verdadero. Lo correcto es `if (x == 5)`.
- Olvidar las llaves `{}` cuando el bloque tiene más de una instrucción.

5.6.3. Repetición: `while`

La condición se evalúa **antes** de cada iteración. Si es falsa desde el principio, el cuerpo no se ejecuta.

```c
while (condición) {
    // cuerpo del bucle
}
```

```c
int contador = 1;
while (contador <= 10) {
    printf("%d\n", contador);
    contador++;
}
```

5.6.4. Repetición: `do-while`

El cuerpo se ejecuta **al menos una vez**; la condición se evalúa al final.

```c
do {
    // cuerpo del bucle
} while (condición);
```

```c
int valor;
do {
    printf("Ingrese un numero positivo: ");
    scanf("%d", &valor);
} while (valor <= 0);
```

Atención

Ojo al traducir desde PSeInt: el `do-while` de C repite *mientras* la condición es **verdadera**, pero el `Repetir ... Hasta Que` de PSeInt repite *hasta que* la condición es verdadera (o sea, mientras es **falsa**). Por eso, al pasar de uno a otro hay que **negar la condición**. El ejemplo anterior en PSeInt sería `Repetir ... Hasta Que valor > 0`, y en C queda `while (valor <= 0)`.

5.6.5. Repetición: `for`

Forma compacta de escribir un bucle cuando se conoce de antemano la cantidad de iteraciones.

```c
for (inicialización; condición; actualización) {
    // cuerpo del bucle
}
```

```c
for (int i = 1; i <= 10; i++) {
    printf("%d\n", i);
}
```

Equivale exactamente a:

```c
int i = 1;
while (i <= 10) {
    printf("%d\n", i);
    i++;
}
```

5.7. Funciones

Una **función** es un bloque de código con nombre que realiza una tarea concreta. Se escribe una sola vez y después se la **llama** (se la usa) desde cualquier parte del programa, todas las veces que haga falta. Ya venimos usando funciones sin darnos cuenta: `main` es una función, y `printf` y `scanf` también.

Las funciones sirven para:

- **No repetir código:** si una tarea se hace en varios lugares, se escribe una vez dentro de una función.
- **Dividir un problema grande en partes chicas**, cada una con un nombre que dice qué hace.
- **Hacer el programa más fácil de leer**: `mostrar(numero)` se entiende mucho mejor que las líneas que hay adentro.

5.7.1. Prototipo y definición

Una función se escribe en dos partes:

- El **prototipo** (o *declaración*): una sola línea, terminada en punto y coma, que dice **qué** hace la función desde afuera: cómo se llama, qué datos recibe y qué tipo de dato devuelve. Es como el "contrato" de la función.
- La **definición**: el código completo, con las instrucciones entre llaves, que dice **cómo** lo hace.

```c
// PROTOTIPO: recibe dos int, devuelve un int
int sumar(int a, int b);

// DEFINICION: la misma primera linea, mas el cuerpo
int sumar(int a, int b) {
    int resultado = a + b;
    return resultado;
}
```

El compilador lee el programa de arriba hacia abajo, y para poder usar una función tiene que conocerla antes. Por eso vamos a ordenar siempre los programas así:

1. Arriba, los **prototipos** de todas las funciones.
2. Después, `main()`, que las usa.
3. Al final, las **definiciones**.

Así, leyendo solo el principio del archivo se ve qué funciones tiene el programa y cómo se usan, sin tener que entrar en los detalles de cada una.

5.7.2. Partes de una función

- **Tipo de retorno** (`int`): el tipo del valor que la función devuelve.
- **Nombre** (`sumar`): se usa para llamarla. Sigue las mismas reglas que los nombres de variables.
- **Parámetros** (`int a, int b`): los datos que la función recibe para trabajar. Cada uno lleva su tipo.
- **Cuerpo** (solo en la definición): las instrucciones, entre llaves.
- **`return`**: termina la función y devuelve el resultado a quien la llamó.

Para **llamarla**, se escribe su nombre con los valores entre paréntesis; el valor devuelto se puede guardar en una variable o usar directamente:

```c
int total = sumar(3, 4);          // total vale 7
printf("%d\n", sumar(10, 5));     // muestra 15
```

5.7.3. Funciones que no devuelven nada: `void`

Si una función hace algo pero no tiene que devolver un resultado, su tipo de retorno es `void` ("vacío"). Si además no recibe parámetros, entre los paréntesis también se pone `void`:

```c
void separador(void);             // no recibe nada, no devuelve nada
void mostrar_doble(int x);        // recibe un dato, no devuelve nada

void separador(void) {
    printf("----------\n");
}

void mostrar_doble(int x) {
    printf("El doble es %d\n", 2 * x);
}
```

5.7.4. Ejemplo completo

```c
#include <stdio.h>

// ---------- Prototipos ----------
int mayor(int a, int b);          // devuelve el mayor de dos numeros

// ---------- Programa principal ----------
int main(void) {
    int x, y;
    printf("Ingrese dos numeros: ");
    scanf("%d %d", &x, &y);
    printf("El mayor es %d\n", mayor(x, y));
    return 0;
}
```

```c

// ---------- Definiciones ----------
int mayor(int a, int b) {
    if (a > b) {
        return a;
    } else {
        return b;
    }
}
```

Nota

Variables locales: los parámetros y las variables declaradas dentro de una función solo existen dentro de esa función. La función recibe una **copia** de los valores: si los modifica, la variable original no cambia.

Importante

Pensando en programas más grandes: separar prototipos y definiciones no es un capricho. Cuando un programa crece, se divide en varios archivos: los prototipos se pasan a un archivo de cabecera (`.h`) y las definiciones a un archivo `.c`. Es exactamente lo que hacen las bibliotecas que ya usamos: `<stdio.h>` contiene los prototipos de `printf` y `scanf`, y por eso hay que incluirlo antes de usarlas. Si desde ahora escribimos los programas con este orden, ese paso va a ser natural.

5.8. Constantes y macros: `#define`

Las líneas que empiezan con # (como `#include`) no son instrucciones de C: son órdenes para el **preprocesador**, un paso que se ejecuta *antes* de compilar. La directiva `#define` le dice al preprocesador que **reemplace un nombre por un texto** en todo el programa.

5.8.1. Constantes

```c
#define LIMITE   10
#define PI       3.1416

for (int i = 0; i < LIMITE; i++) { ... }   // queda: i < 10
```

Así, los "números mágicos" del programa tienen un nombre que explica qué son, y si hay que cambiarlos se cambian en un solo lugar. Por convención, los nombres definidos con `#define` se escriben en **MAYÚSCULAS**.

Atención

Un `#define` **no lleva** signo = **ni** punto y coma al final: el preprocesador copiaría también esos caracteres en cada lugar donde se use el nombre.

5.8.2. Macros con parámetros

Un `#define` también puede recibir parámetros. Se llama **macro** y se usa parecido a una función, pero en realidad es un *reemplazo de texto*: el preprocesador pega el texto de la macro en cada lugar donde se usa, reemplazando los parámetros.

```c
#define CUADRADO(x)   ((x) * (x))

int a = CUADRADO(5);        // ((5) * (5))          -> 25
int b = CUADRADO(2 + 1);    // ((2 + 1) * (2 + 1)) -> 9
```

Importante

Siempre entre paréntesis. Como la macro es un reemplazo de texto, hay que poner entre paréntesis cada parámetro y la expresión completa. Si se hubiera escrito `#define CUADRADO(x) x * x`, entonces `CUADRADO(2 + 1)` se reemplaza por `2 + 1 * 2 + 1`, que da **5** y no 9.

Las macros son muy usadas en la programación de microcontroladores: en el próximo bloque vamos a definir macros para poner en 1, poner en 0 y leer bits de los registros.

5.9. Equivalencia entre pseudocódigo y C

Pseudocódigo (PSeInt)	C
`Leer variable`	`scanf("%d", &variable);`
`Escribir "texto"`	`printf("texto\n");`
`Escribir variable`	`printf("%d\n", variable);`
`variable ← valor`	`variable = valor;`
`Si condición Entonces ... Fin Si`	`if (condición) { ... }`
`Si ... Sino ... Fin Si`	`if (...) { ... } else { ... }`
`Mientras condición Hacer ... Fin Mientras`	`while (condición) { ... }`
`Repetir ... Hasta Que condición`	`do { ... } while (!(condición));`
`MOD`	`%`
`Y`	`&&`
`O`	`||`
`NO`	`!`

5.10. Ejercicios: los algoritmos del TP en C

Traducir a C los algoritmos de los ejercicios del Bloque 4, partiendo de los diagramas de flujo hechos en PSeInt. Cada programa se debe compilar y probar con distintos valores de entrada, verificando que se comporte igual que el diagrama (se puede usar cualquier compilador de C estándar; ver la sección siguiente).

1. **Suma de dos valores:** sumar 2 valores e informar el resultado.
2. **Operación con tres valores:** ingresar 3 valores, sumar los dos primeros, restarle el tercero e informar el resultado.
3. **Módulo de un valor:** calcular el módulo (valor absoluto) de un número, sin usar funciones de biblioteca.
4. **Sensor de temperatura:** informar "hace calor" si la temperatura supera los 26°C, "hace frío" si es menor a 22°C, y no informar nada en otro caso.
5. **Promedio de una serie de valores:** calcular el promedio de todos los números ingresados; el ingreso termina con un 0.

Nota

Para tener en cuenta al pasar a C:

- Elegir el tipo de cada variable: si un valor puede tener decimales (una temperatura, un promedio), tiene que ser `float`, y en `scanf`/`printf` se usa `%f` en lugar de `%d`.
- Cada estructura del diagrama tiene su equivalente en C (ver la tabla de la sección anterior). Ojo con *repetir–hasta*: hay que negar la condición.

5.11. Entorno de desarrollo

Para compilar y ejecutar programas en C se puede usar cualquiera de los siguientes entornos:

Online (sin instalación):

- `replit.com` — crea un proyecto de tipo C.
- `onlinegdb.com` — compilador + depurador online.
- `tutorialspoint.com/compile_c_online.php`

Local:

- **VSCode** + extensión C/C++ + compilador GCC (a través de MinGW en Windows).

Capítulo 6

Programación de Microcontroladores en C

6.1. C estándar vs. C embebido

En el Bloque 5 aprendimos C para correr programas en una PC. Al programar un microcontrolador, el lenguaje es el mismo pero el contexto cambia completamente:

	C en PC	**C en microcontrolador**
Sistema operativo	Sí (Windows, Linux)	No hay
Entrada/salida	Teclado, pantalla (`printf`, `scanf`)	Pines físicos del chip
Memoria	Abundante (GB)	Escasa (KB)
Punto de entrada	`main()` retorna cuando termina	`main()` nunca debe retornar — bucle infinito
Bibliotecas estándar	`<stdio.h>`, `<math.h>`, etc.	Reemplazadas por bibliotecas del fabricante

En lugar de `#include <stdio.h>` usaremos las bibliotecas de AVR:

```
#include <avr/io.h>         // definiciones de registros y pines del ATmega
#include <util/delay.h>     // función _delay_ms() para retardos
#include <avr/interrupt.h> // manejo de interrupciones
```

6.1.1. Estructura básica de un programa embebido

```
#include <avr/io.h>
#include <util/delay.h>

int main(void) {

    // --- INICIALIZACIÓN (se ejecuta una sola vez al encender) ---
    // Configurar puertos, registros, periféricos...

    // --- BUCLE PRINCIPAL (se ejecuta para siempre) ---
    while (1) {
        // lógica de la aplicación
    }

    return 0; // nunca se llega aquí, pero el compilador lo requiere
}
```

Esta estructura de **inicialización + bucle infinito** es el patrón fundamental de todo programa embebido. El microcontrolador no tiene un sistema operativo que lo controle: es el programa quien toma el control total del hardware desde el primer momento.

6.2. Los puertos de E/S del ATmega328

El ATmega328 tiene tres puertos de E/S digitales: **PORTB**, **PORTC** y **PORTD**, cada uno con hasta 8 pines. Cada puerto se controla mediante tres registros de 8 bits:

Registro	Función
`DDRx`	*Data Direction Register* — define si cada pin es **entrada (0)** o **salida (1)**
`PORTx`	Si el pin es **salida**: escribe el nivel lógico (0 = LOW, 1 = HIGH). Si el pin es **entrada**: activa (1) o desactiva (0) la resistencia de pull-up interna
`PINx`	*Port Input Pins* — permite **leer** el estado actual de los pines (solo lectura)

Donde `x` es `B`, `C` o `D` según el puerto.

▶ **Diagrama generado en CircuiTikZ — revisar antes de la versión final**

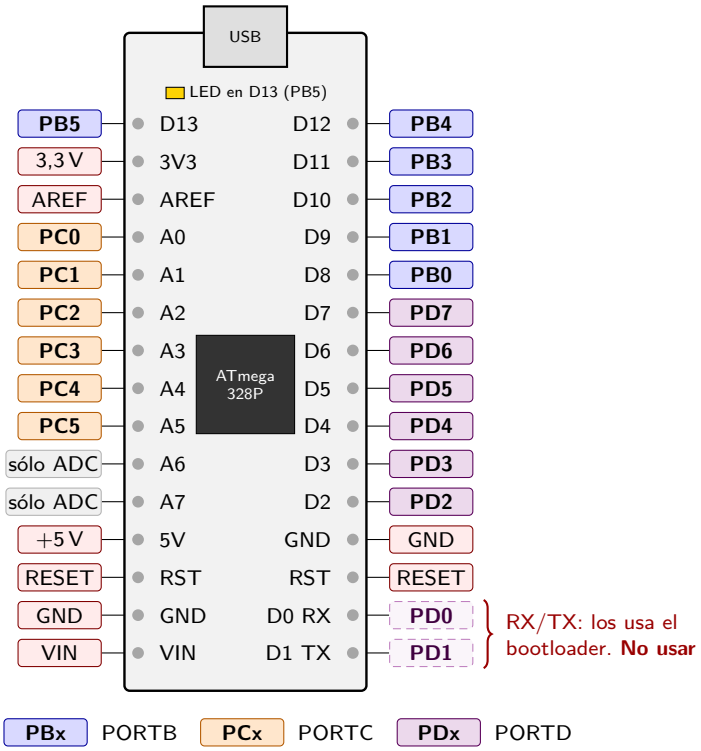


Figura 6.1: Mapa de pines del Arduino Nano y su correspondencia con los puertos del ATmega328P (PB, PC, PD)

Las etiquetas D0–D13 y A0–A5 son las que vienen impresas en la placa Arduino: sirven solo para **ubicar físicamente** cada pin. En nuestros programas nunca usamos esa numeración, sino siempre el **puerto** y el **bit** del ATmega (por ejemplo, el LED de la placa está en el bit 5 del puerto B: `PB5`).

Atención

No usar PD0 (RX) ni PD1 (TX). Estos dos pines están conectados al conversor USB–serie de la placa y los usa el *bootloader* para recibir el programa cuando lo cargamos desde la PC. Si les conectamos un LED, un pulsador o un display, la carga del programa puede fallar o el circuito puede hacer cosas raras mientras se programa. Por eso, en el curso **se dejan siempre libres**: del puerto D solo usamos de PD2 a PD7.

6.3. Operadores de manejo de bits

Para configurar y controlar los registros del microcontrolador necesitamos modificar bits individuales sin afectar los demás. Para eso usamos los **operadores bitwise** (a nivel de bits):

Operador	Nombre	Operación
&	AND bit a bit	El bit resultado es 1 solo si ambos operandos tienen 1
|	OR bit a bit	El bit resultado es 1 si al menos uno de los operandos tiene 1
^	XOR bit a bit	El bit resultado es 1 si los operandos son distintos
~	NOT bit a bit	Invierte todos los bits
«	Desplazamiento a la izquierda	Desplaza los bits hacia la izquierda N posiciones
»	Desplazamiento a la derecha	Desplaza los bits hacia la derecha N posiciones

6.3.1. Desplazamiento de bits

El operador « es clave para crear **máscaras**: valores que tienen un único bit en 1 en la posición deseada.

```
1 << 0   ->   0b00000001   ->   0x01   (bit 0)
1 << 1   ->   0b00000010   ->   0x02   (bit 1)
1 << 3   ->   0b00001000   ->   0x08   (bit 3)
1 << 7   ->   0b10000000   ->   0x80   (bit 7)
```

En la biblioteca `<avr/io.h>` los bits de cada registro ya tienen nombres definidos. Por ejemplo, `PD3` es el bit 3 del puerto D, y su valor es 3. Entonces `(1 « PD3)` genera la máscara para ese bit.

6.4. Técnicas de enmascarado

6.4.1. Poner un bit en 1 (SET)

Se usa OR con una máscara que tiene 1 en la posición deseada:

```
PORTD = PORTD | (1 << PD3);    // pone el bit 3 de PORTD en 1
PORTD |= (1 << PD3);            // forma abreviada (equivalente)
```

Ejemplo: `PORTD = 0b00000000`, aplicar máscara `0b00001000`:

```
  0 0 0 0 0 0 0 0   (PORTD original)
| 0 0 0 0 1 0 0 0   (máscara: 1 << PD3)
= 0 0 0 0 1 0 0 0   (resultado: solo el bit 3 cambió)
```

6.4.2. Poner un bit en 0 (CLEAR)

Se usa AND con el complemento de la máscara (~):

```
PORTD = PORTD & ~(1 << PD3);   // pone el bit 3 de PORTD en 0
PORTD &= ~(1 << PD3);           // forma abreviada
```

Ejemplo: `PORTD = 0b11111111`, aplicar máscara:

```
  1 1 1 1 1 1 1 1   (PORTD original)
& 1 1 1 1 0 1 1 1   (~(1 << PD3): todos 1 excepto el bit 3)
= 1 1 1 1 0 1 1 1   (resultado: solo el bit 3 se puso en 0)
```

6.4.3. Invertir un bit (TOGGLE)

Se usa XOR con la máscara:

```
PORTD ^= (1 << PD3);    // invierte el bit 3 de PORTD
```

6.4.4. Leer el estado de un bit

Se usa AND con la máscara y se compara con cero:

```
if (PIND & (1 << PD2)) {
    // el bit 2 de PIND está en 1 (pin en HIGH)
}

if (!(PIND & (1 << PD2))) {
    // el bit 2 de PIND está en 0 (pin en LOW)
}
```

6.4.5. Macros para manejar bits

Las expresiones anteriores se repiten todo el tiempo y es fácil equivocarse con un ~ o un paréntesis. Por eso conviene definirlas una sola vez como **macros** (ver Bloque 5), que reciben el **registro** y el **número de bit**:

```
// pone el bit en 1 / lo pone en 0 / lo invierte
#define SET_BIT(reg, bit)     ((reg) |=  (1 << (bit)))
#define CLR_BIT(reg, bit)     ((reg) &= ~(1 << (bit)))
#define TOGGLE_BIT(reg, bit)  ((reg) ^=  (1 << (bit)))
```

Se usan así:

```
SET_BIT(DDRD, PD5);       // PD5 como salida
SET_BIT(PORTD, PD5);      // enciende el LED en PD5
CLR_BIT(PORTD, PD5);      // apaga el LED
TOGGLE_BIT(PORTB, PB5);   // invierte el LED de la placa
```

El preprocesador reemplaza, por ejemplo, `SET_BIT(PORTD, PD5)` por `((PORTD) |= (1 << (PD5)))`: es exactamente el mismo código de antes, pero mucho más fácil de leer y de escribir.

6.4.5.1. Leer entradas: `bit_is_set` y `bit_is_clear`

Para leer bits, la biblioteca `avr-libc` ya trae macros listas. Como vienen incluidas en `<avr/io.h>`, no hace falta definirlas:

Macro	Qué hace
`bit_is_set(reg, bit)`	Verdadero (distinto de 0) si el bit está en **1**
`bit_is_clear(reg, bit)`	Verdadero si el bit está en **0**
`loop_until_bit_is_set(reg, bit)`	Espera (bloqueando) hasta que el bit se ponga en 1
`loop_until_bit_is_clear(reg, bit)`	Espera (bloqueando) hasta que el bit se ponga en 0

Para leer un pin de entrada se usa siempre el registro `PINx`:

```
if (bit_is_clear(PIND, PD2)) {
    // PD2 en 0: con pull-up, el pulsador esta presionado
}

if (bit_is_set(PIND, PD2)) {
    // PD2 en 1: el pulsador esta suelto
}

loop_until_bit_is_clear(PIND, PD2);  // espera a que se presione
```

Nota

`bit_is_clear(PIND, PD2)` es lo mismo que `!(PIND & (1 << PD2))`, pero se lee casi como una frase: "si el bit PD2 de PIND está en cero". Las macros `loop_until_...` son **bloqueantes** (el programa no hace nada más mientras espera), así que solo conviene usarlas en programas simples.

6.5. Configuración de puertos (TP1)

6.5.1. Configurar un pin como salida

Para usar un pin como salida, se pone en 1 el bit correspondiente en `DDRx`:

```
DDRD |= (1 << PD5);    // PD5 como salida
```

6.5.2. Configurar un pin como entrada

Para usar un pin como entrada, se pone en 0 el bit correspondiente en `DDRx`:

```
DDRD &= ~(1 << PD2);   // PD2 como entrada
```

6.5.3. Activar la resistencia de pull-up interna

Cuando un pin es entrada, se puede activar su resistencia pull-up interna poniéndolo en 1 en `PORTx`. Esto evita que el pin "flote" cuando no hay nada conectado:

```
DDRD  &= ~(1 << PD2);  // PD2 como entrada
PORTD |=  (1 << PD2);  // activa pull-up interno en PD2
```

Con pull-up activo, la lógica se **invierte**: el pin lee 1 cuando el pulsador está suelto y 0 cuando está presionado (porque el pulsador conecta el pin a GND).

6.5.4. Escribir en un pin de salida

```
PORTD |=  (1 << PD5);  // pone PD5 en HIGH (LED encendido)
PORTD &= ~(1 << PD5);  // pone PD5 en LOW  (LED apagado)
```

6.5.5. Leer un pin de entrada

```
if (!(PIND & (1 << PD2))) {
    // pulsador presionado (con pull-up: pulsador conecta a GND -> lee 0)
}
```

6.5.6. Ejemplo completo — TP1: un pulsador controla un LED

```
#include <avr/io.h>

int main(void) {

    // Inicialización
    DDRD |=  (1 << PD5);   // PD5 como salida -> LED
    DDRD &= ~(1 << PD2);   // PD2 como entrada -> pulsador
    PORTD |= (1 << PD2);   // activar pull-up en PD2

    while (1) {
        if (!(PIND & (1 << PD2))) {
            // pulsador presionado -> encender LED
            PORTD |= (1 << PD5);
        } else {
            // pulsador suelto -> apagar LED
            PORTD &= ~(1 << PD5);
        }
    }

    return 0;
}
```

El mismo programa, escrito con las macros de la sección anterior:

```
#include <avr/io.h>

#define SET_BIT(reg, bit)  ((reg) |=  (1 << (bit)))
#define CLR_BIT(reg, bit)  ((reg) &= ~(1 << (bit)))

int main(void) {
    SET_BIT(DDRD, PD5);    // LED como salida
    CLR_BIT(DDRD, PD2);    // pulsador como entrada
    SET_BIT(PORTD, PD2);   // pull-up en el pulsador

```

```
    while (1) {
        if (bit_is_clear(PIND, PD2)) {    // pulsador presionado
            SET_BIT(PORTD, PD5);          // encender LED
        } else {
            CLR_BIT(PORTD, PD5);          // apagar LED
        }
    }

    return 0;
}
```

6.6. Retardos

La función `_delay_ms(n)` detiene la ejecución durante `n` milisegundos. Es un retardo **bloqueante**: mientras espera, el programa no hace nada más.

```
#include <util/delay.h>

_delay_ms(500);     // espera 500 ms
_delay_us(100);     // espera 100 microsegundos
```

Importante

Para que `_delay_ms` funcione correctamente, el compilador debe saber a qué frecuencia corre el microcontrolador. Esto se configura con la constante `F_CPU` antes del include, o en las opciones del compilador:

```
#define F_CPU 16000000UL    // 16 MHz (Arduino Nano)
#include <util/delay.h>
```

Los retardos son útiles para efectos simples (parpadeo, secuencias de LEDs), pero tienen una desventaja importante: **bloquean todo el programa**. Mientras espera, el microcontrolador no puede leer pulsadores, actualizar displays ni hacer otra cosa. Para aplicaciones más complejas se usan los **timers con interrupciones** (TP5).

6.7. Escribir en el puerto completo

Cuando se controlan varios pines simultáneamente (por ejemplo, LEDs conectados a todo un puerto), conviene escribir en el registro completo en lugar de bit a bit:

```
PORTB = 0b00110101;    // escribe el valor completo en el puerto B
PORTB = 0x3F;          // lo mismo, pero en hexadecimal
```

Atención

Esto sobreescribe **todos** los bits del puerto. Si algunos pines del puerto se usan para otra cosa (entradas con pull-up, o PD0/PD1 en el puerto D), también se modifican.

Por eso, cuando no todos los bits del puerto son "nuestros", hay que escribir solo algunos bits, enmascarando.

6.7.1. Escribir solo algunos bits: enmascarar

Supongamos que queremos mandar los 4 bits altos de una variable a PD4..PD7 sin tocar PD0..PD3 (que pueden estar usados para otra cosa, como PD0 y PD1). Se hace en una sola línea, combinando AND y OR (técnica de **lectura–modificación–escritura**):

```
PORTD = (PORTD & 0b00001111) | (variable & 0b11110000);
```

1. `PORTD & 0b00001111`: **conserva** los bits del puerto que no son nuestros (PD0..PD3) y pone en 0 los que vamos a escribir (PD4..PD7).
2. `variable & 0b11110000`: se queda solo con los bits de la variable que queremos mandar (4..7) y descarta el resto.
3. El **OR** (`|`) junta las dos partes y el resultado se escribe en el puerto.

```
  0 1 0 1 1 0 1 0   (PORTD actual: PD0..PD3 se deben conservar)
& 0 0 0 0 1 1 1 1   (mascara 0b00001111)
= 0 0 0 0 1 0 1 0   -> parte que se conserva

  1 0 0 1 0 1 1 0   (variable)
& 1 1 1 1 0 0 0 0   (mascara 0b11110000)
= 1 0 0 1 0 0 0 0   -> parte nueva

  0 0 0 0 1 0 1 0
| 1 0 0 1 0 0 0 0
= 1 0 0 1 1 0 1 0   -> nuevo PORTD (solo cambiaron PD4..PD7)
```

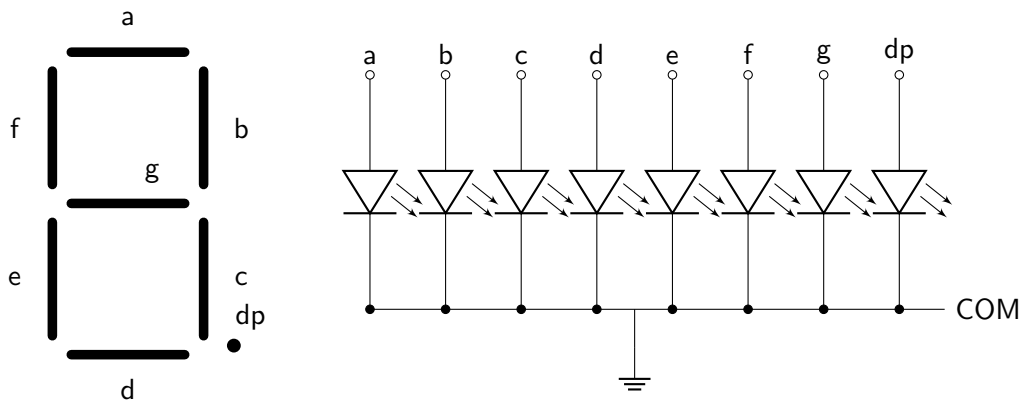
Nota

Las dos máscaras son siempre **complementarias**: donde una tiene 1, la otra tiene 0. La primera marca qué bits del puerto se conservan; la segunda, qué bits de la variable se escriben.

6.8. Display de 7 segmentos (TP4)

Un display de 7 segmentos tiene 7 LEDs (a, b, c, d, e, f, g) y opcionalmente un punto decimal (dp), que se encienden en distintas combinaciones para mostrar un dígito.

▶ **Diagrama generado en CircuiTikZ — revisar antes de la versión final**



Disposición de los segmentos *Cátodo común: COM a GND*

Figura 6.2: Disposición de los segmentos a–g en un display de 7 segmentos de cátodo común

Existen dos tipos:

- **Ánodo común:** el cátodo de cada segmento se conecta al pin del microcontrolador. Para encender un segmento se pone el pin en **LOW (0)**.
- **Cátodo común:** el ánodo de cada segmento se conecta al pin del microcontrolador. Para encender un segmento se pone el pin en **HIGH (1)**.

Para cada dígito se arma un **patrón**: un byte donde cada bit indica si un segmento se enciende (1) o no (0). Usamos siempre el mismo orden: el bit 0 es el segmento a, el bit 1 el b, y así hasta el bit 6, que es el g. Para cátodo común, la tabla de patrones de los dígitos 0 al 9 es:

Dígito	g	f	e	d	c	b	a	Hex
0	0	1	1	1	1	1	1	0x3F
1	0	0	0	0	1	1	0	0x06
2	1	0	1	1	0	1	1	0x5B
3	1	0	0	1	1	1	1	0x4F
4	1	1	0	0	1	1	0	0x66
5	1	1	0	1	1	0	1	0x6D
6	1	1	1	1	1	0	1	0x7D
7	0	0	0	0	1	1	1	0x07
8	1	1	1	1	1	1	1	0x7F
9	1	1	0	1	1	1	1	0x6F

Cuadro 6.1: Patrones de display 7 segmentos — cátodo común, bit 0 = a . . . bit 6 = g

6.8.1. Conectar el display: los segmentos en dos puertos

Qué pin del microcontrolador va a cada segmento lo decide cada uno al armar el circuito. Pero hay una restricción: como **PD0 y PD1 no se usan**, al puerto D le quedan solo 6 pines libres, y el display tiene 7 segmentos. Entonces, los segmentos **no entran en un solo puerto** y hay que repartirlos entre dos. Para escribir un patrón se usa la técnica de enmascarar de la sección anterior, una vez en cada puerto.

La forma más simple es conectar cada segmento al **mismo número de bit** que tiene en el patrón. Así no hay que desplazar nada: alcanza con elegir, en cada puerto, qué bits del patrón le tocan. Por ejemplo (una conexión posible):

Segmento	a	b	c	d	e	f	g
Bit del patrón	0	1	2	3	4	5	6
Pin	PB0	PB1	PB2	PB3	PD4	PD5	PD6

Cuadro 6.2: Una conexión posible: a–d en el puerto B y e–g en el puerto D, cada segmento en el mismo bit que en el patrón.

Con esa conexión, mostrar un patrón son dos líneas: los bits 0..3 van al puerto B y los bits 4..6 al puerto D, conservando en cada puerto los bits que no son del display:

```
PORTB = (PORTB & 0b11110000) | (patron & 0b00001111); // a..d
PORTD = (PORTD & 0b10001111) | (patron & 0b01110000); // e..g
```

Nota

Si tu conexión es otra, la tabla de patrones **no cambia**: cambian solo las máscaras (y, si un segmento no quedó en el mismo número de bit que en el patrón, hay que desplazarlo con << o >>).

En el código se almacena la tabla en un arreglo, se indexa con el dígito a mostrar y se usa una **función** (ver Bloque 5), `mostrar()`, que lleva el patrón a los pines. Así, las máscaras se escriben una sola vez y el resto del programa solo llama a `mostrar(digitos[n])`:

```
#include <avr/io.h>

// Patrones para dígitos 0-9 (cátodo común, bit 0 = a ... bit 6 = g)
const uint8_t digitos[10] = {
    0x3F, // 0
    0x06, // 1
    0x5B, // 2
    0x4F, // 3
    0x66, // 4
    0x6D, // 5
    0x7D, // 6
    0x07, // 7
    0x7F, // 8
    0x6F  // 9
};
```

```c

// ---------- Prototipos ----------
void mostrar(uint8_t patron);   // lleva un patrón a los segmentos

// ---------- Programa principal ----------
int main(void) {
    DDRB |= 0b00001111;   // PB0..PB3 como salidas (segmentos a..d)
    DDRD |= 0b01110000;   // PD4..PD6 como salidas (segmentos e..g)
    mostrar(digitos[0]);  // mostrar el dígito 0

    while (1) {
        // la lógica de conteo iría aquí
    }
    return 0;
}

// ---------- Definiciones ----------
// Conexión de ejemplo: a..d en PB0..PB3, e..g en PD4..PD6
void mostrar(uint8_t patron) {
    PORTB = (PORTB & 0b11110000) | (patron & 0b00001111);
    PORTD = (PORTD & 0b10001111) | (patron & 0b01110000);
}
```

6.9. Timers e interrupciones (TP5)

6.9.1. El problema del código bloqueante

Con `_delay_ms()` el microcontrolador se queda "dormido" esperando. No puede atender pulsadores ni actualizar displays mientras espera. Esto es el **código bloqueante**.

La solución son los **timers con interrupciones**: el timer cuenta en segundo plano y cuando llega a un valor configurado, interrumpe al programa principal para ejecutar una función especial (ISR).

6.9.2. Interrupciones

Una **interrupción** es una señal que detiene temporalmente la ejecución del programa principal para atender un evento (el timer llegó al valor, llegó un dato por UART, cambió un pin, etc.). Al terminar la función de interrupción, el programa principal continúa desde donde fue interrumpido.

Para usar interrupciones:

1. Configurar la fuente de interrupción (timer, pin externo, etc.).
2. Definir la función ISR (*Interrupt Service Routine*) que se ejecutará.
3. Habilitar las interrupciones globales con `sei()`.

```c
#include <avr/io.h>
#include <avr/interrupt.h>

// Función ISR: se ejecuta automáticamente cuando ocurre la interrupción
ISR(TIMER0_COMPA_vect) {
    // código que se ejecuta cada 1 ms (modo CTC, comparación con OCR0A)
}

int main(void) {
    // configurar el timer...

    sei();    // habilitar interrupciones globales

    while (1) {
        // programa principal -- sin delays bloqueantes
    }
    return 0;
}
```

La función ISR **no se llama** desde el programa: la llama automáticamente el hardware cuando ocurre el evento. El argumento de `ISR` es el **vector de interrupción** que indica la fuente.

6.9.3. Vectores de interrupción del ATmega328

Los principales vectores disponibles son:

```c
INT0_vect          // Interrupción externa en pin INT0 (PD2)
INT1_vect          // Interrupción externa en pin INT1 (PD3)
PCINT0_vect        // Cambio de pin en PORTB
PCINT1_vect        // Cambio de pin en PORTC
PCINT2_vect        // Cambio de pin en PORTD
TIMER0_OVF_vect    // Desbordamiento del Timer 0
TIMER0_COMPA_vect  // Comparación A del Timer 0
TIMER1_OVF_vect    // Desbordamiento del Timer 1
TIMER1_COMPA_vect  // Comparación A del Timer 1
TIMER2_OVF_vect    // Desbordamiento del Timer 2
USART_RX_vect      // Recepción UART completa
ADC_vect           // Conversión ADC completa
TWI_vect           // I²C (Two Wire Interface)
SPI_STC_vect       // SPI transferencia completa
```

6.9.4. Configuración del Timer 0 en modo CTC

El modo CTC (*Clear Timer on Compare Match*) hace que el contador se reinicie automáticamente a 0 cada vez que alcanza el valor cargado en el registro `OCR0A`, generando la interrupción `TIMER0_COMPA_vect`. Esto permite obtener intervalos de tiempo exactos y predecibles sin depender del período de desbordamiento.

El valor de `OCR0A` se calcula con la fórmula:

$$\text{OCR0A} = \frac{F_{\text{CPU}}}{\text{prescaler} \times f_{\text{interrupción}}} - 1$$

Para obtener una interrupción cada **1 ms** (1000 Hz) con el ATmega a 16 MHz y preescalador 64:

$$OCR0A = \frac{16.000.000}{64 \times 1000} - 1 = 250 - 1 = 249$$

```
// Configurar Timer 0 en modo CTC -- interrupción cada 1 ms
// F_CPU = 16 MHz, preescalador = 64, OCR0A = 249
TCCR0A = (1 << WGM01);                  // modo CTC
TCCR0B = (1 << CS01) | (1 << CS00);    // preescalador 64
OCR0A  = 249;                           // comparación: interrupt
    cada 1 ms

// Habilitar interrupción por comparación A
TIMSK0 = (1 << OCIE0A);
```

En modo CTC el contador no desborda: sube de 0 a `OCR0A`, genera la interrupción y vuelve a 0. El período de la interrupción es exactamente 1 ms, lo que hace muy simple llevar cuentas de tiempo en la ISR (cada llamada = 1 ms).

6.9.5. Ejemplo — parpadeo de LED sin código bloqueante

La ISR se ejecuta cada 1 ms. Contando 500 llamadas se obtiene un período de 500 ms → el LED parpadea a 1 Hz (500 ms encendido, 500 ms apagado).

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h>

volatile uint16_t contador_ms = 0;

ISR(TIMER0_COMPA_vect) {            // se ejecuta cada 1 ms
    contador_ms++;
    if (contador_ms >= 500) {       // 500 ms
        contador_ms = 0;
        PORTB ^= (1 << PB5);        // invierte el LED de la placa (
    PB5)
    }
}

int main(void) {
    DDRB |= (1 << PB5);             // PB5 (LED de la placa) como
    salida

    // Timer 0 -- modo CTC -- 1 ms
    TCCR0A = (1 << WGM01);
    TCCR0B = (1 << CS01) | (1 << CS00);    // preescalador 64
    OCR0A  = 249;
    TIMSK0 = (1 << OCIE0A);
```

```
23
24      sei();
25
26      while (1) {
27          // libre para hacer otras cosas (sin bloqueos)
28      }
29      return 0;
30  }
```

Nota

La palabra clave `volatile`: cuando una variable se modifica dentro de una ISR y se usa fuera de ella, **debe declararse `volatile`**. Esto le indica al compilador que no optimice los accesos a esa variable, ya que puede cambiar en cualquier momento por la interrupción.

6.10. Multiplexado de displays (TP6)

Cuando se usan varios displays de 7 segmentos, conectar cada uno con 7 pines propios requeriría demasiados pines del microcontrolador. La solución es el **multiplexado**: compartir los 7 pines de segmentos entre todos los displays y activar un display a la vez, rotando muy rápido (>50 Hz) para que el ojo humano perciba todos encendidos simultáneamente.

6.10.1. Conexión de los displays

Los segmentos de los 4 displays se conectan en paralelo (el segmento a de todos los displays al mismo pin, el b al mismo pin, etc.), cada línea con su resistencia limitadora. Para elegir qué dígito se enciende se maneja el **común** de cada display.

Hay que tener en cuenta que por el común pasa la corriente de *todos* los segmentos encendidos del dígito (hasta 8, contando el punto), y que un pin del ATmega328 soporta como máximo **40 mA**. Cómo se conecta el común depende del tipo de display.

6.10.1.1. Cátodo común: con transistor NPN

Con un display de cátodo común, la corriente de los segmentos tiene que *salir* del display por el común hacia GND. Cada común se conecta a GND a través de un **transistor NPN**, que el micro maneja con una **resistencia de base** de alrededor de 1 kΩ. Así el pin solo entrega la pequeña corriente de base, y la corriente de los segmentos la maneja el transistor.

▶ **Diagrama generado en CircuiTikZ — revisar antes de la versión final**

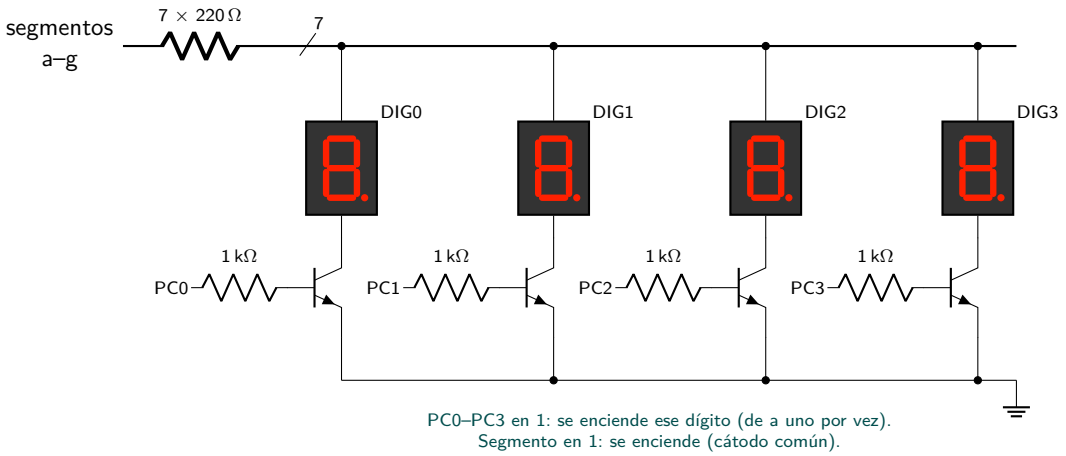


Figura 6.3: Multiplexado de 4 displays de cátodo común: segmentos compartidos (con sus resistencias) y un transistor NPN por dígito, manejado desde PC0–PC3.

6.10.1.2. Opción: ánodo común conectado directo al micro

No siempre se consiguen displays de cátodo común. Con uno de **ánodo común** hay una opción mucho más sencilla, sin transistores: cada ánodo se conecta a un pin del micro a través de **una sola resistencia**, y los segmentos van directo a sus pines, sin resistencias.

- El pin de selección **entrega** la corriente al dígito: se enciende poniéndolo en **1**. Los segmentos se encienden con **0**.
- La resistencia del ánodo limita la corriente *total* del dígito, sin importar cuántos segmentos estén encendidos. Con 220 Ω: $(5\,\mathrm{V} - 2\,\mathrm{V})/220\,\Omega \approx 14\,\mathrm{mA}$, bien por debajo de los 40 mA del pin.
- Conviene usar displays de **alto brillo**, que se ven bien con poca corriente.

▶ **Diagrama generado en CircuiTikZ — revisar antes de la versión final**

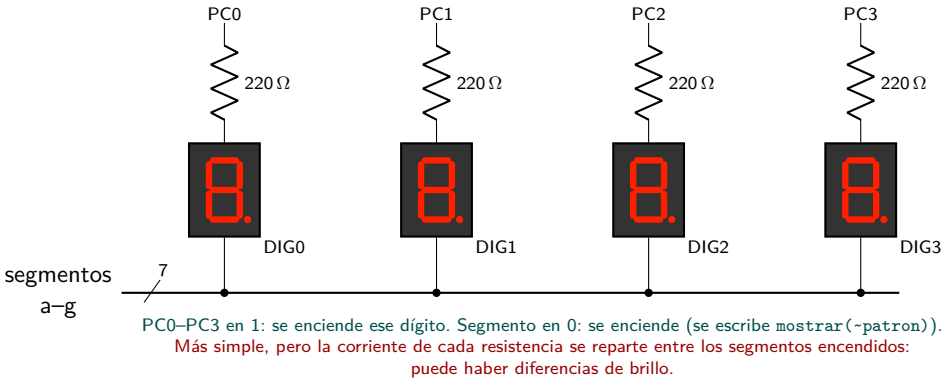


Figura 6.4: Opción con displays de ánodo común: una resistencia por ánodo, segmentos directo al micro. Más simple, pero con posibles diferencias de brillo.

Atención

Diferencias de brillo. Con una sola resistencia por dígito, esa corriente se **reparte** entre los segmentos encendidos: un "1" (2 segmentos) se ve más brillante que un "8" (7 segmentos). Además, los LEDs de cada segmento nunca son idénticos, y el que necesita un poco menos de tensión se lleva más corriente y brilla más. Es una conexión muy práctica para armar rápido, pero el display no se ve parejo.

6.10.1.3. ¿Con transistor o directo? Diferencias, ventajas y problemas

La diferencia de fondo es **quién maneja la corriente del común** y **dónde se limita**. Con transistor, el pin del micro solo le da una orden al transistor (una corriente de base chica) y cada segmento tiene su propia resistencia. Con la conexión directa, la corriente del dígito pasa **por el propio pin del micro** y la limita una sola resistencia, compartida por todos los segmentos.

	Con transistor (cátodo común)	**Directo (ánodo común)**
Corriente por el pin	Solo la de base (pocos mA)	La de todo el dígito (limitada por la resistencia del ánodo)
Resistencias	Una por segmento (7) y una de base por dígito	Una por dígito (4), en el ánodo
Componentes	Además, un transistor por dígito	Ninguno más
Brillo	Parejo: cada segmento tiene su corriente	Desparejo: la corriente se reparte entre los segmentos encendidos
Display	Cualquiera	Conviene de alto brillo
Armado	Más cables y más lugares donde equivocarse	Mucho más simple y rápido

- **Con transistor:** el micro trabaja descansado y el display se ve parejo y con buen brillo, sin importar qué dígito se muestre. **Problema:** más componentes y más conexiones en la protoboard.
- **Directo:** el circuito es mínimo y la resistencia del ánodo protege siempre al pin. **Problema:** el brillo depende de cuántos segmentos estén encendidos y de las diferencias entre los LEDs.

	Cátodo común	**Ánodo común**
Conexión del común	Transistor NPN a GND (con resistencia de base)	Al pin del micro, con una resistencia
Encender un dígito	Pin de selección en **1**	Pin de selección en **1**
Encender un segmento	Pin del segmento en **1**	Pin del segmento en **0**
En el código	`mostrar(patron)`	`mostrar(~patron)`

Nota

Con ánodo común los segmentos se encienden con **0**, así que los patrones de la tabla se escriben **invertidos** con el operador ~ (NOT bit a bit). La tabla de patrones y el resto del programa no cambian.

6.10.2. Principio de funcionamiento

1. Activar display 1: activar su pin de selección, escribir el dígito 1 en los segmentos.
2. Pequeña espera (∼2 ms).
3. Apagar display 1, activar display 2: escribir el dígito 2 en los segmentos.
4. Pequeña espera (∼2 ms).
5. Repetir para displays 3 y 4.
6. Volver al display 1.

Si el ciclo completo (pasar por los 4 displays) dura menos de ∼20 ms, es decir, se repite más de 50 veces por segundo, el parpadeo es imperceptible.

6.10.3. Implementación con Timer 0

El multiplexado se implementa en la ISR del Timer 0, sin bloquear el programa principal:

```c
#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h>

// Patrones de dígitos (cátodo común)
const uint8_t digitos[10] = {
    0x3F, 0x06, 0x5B, 0x4F, 0x66,
    0x6D, 0x7D, 0x07, 0x7F, 0x6F
};

// ---------- Prototipos ----------
void mostrar(uint8_t patron);   // lleva un patrón a los segmentos

// Pines de selección de cada display (en PORTC)
#define SEL0  PC0
#define SEL1  PC1
#define SEL2  PC2
#define SEL3  PC3

volatile uint8_t display_actual = 0;
uint8_t valores[4] = {1, 2, 3, 4};  // dígito de cada display

// ISR cada 1 ms -- rota entre los 4 displays
// Cada display se muestra 1 ms -> refresco = 1/(4 x 1 ms) = 250 Hz
ISR(TIMER0_COMPA_vect) {
    // Apagar todos los displays
    PORTC &= ~((1<<SEL0)|(1<<SEL1)|(1<<SEL2)|(1<<SEL3));

    // Escribir el dígito del display actual en los segmentos
    mostrar(digitos[valores[display_actual]]);

    // Activar solo el display actual
    PORTC |= (1 << (SEL0 + display_actual));

    // Avanzar al siguiente display
    display_actual++;
    if (display_actual >= 4) display_actual = 0;
}

int main(void) {
    DDRB |= 0b00001111;             // PB0..PB3: segmentos a..d
    DDRD |= 0b01110000;             // PD4..PD6: segmentos e..g
    DDRC |= (1<<SEL0)|(1<<SEL1)|(1<<SEL2)|(1<<SEL3); // selección

    // Timer 0 -- modo CTC -- 1 ms
    TCCR0A = (1 << WGM01);
    TCCR0B = (1 << CS01) | (1 << CS00);   // preescalador 64
    OCR0A  = 249;
    TIMSK0 = (1 << OCIE0A);

    sei();

    while (1) {
```

```
        // el programa principal puede cambiar valores[]
    }
    return 0;
}

// ---------- Definiciones ----------
// Misma conexión que en la sección del display (TP4):
//   a..d en PB0..PB3, e..g en PD4..PD6
void mostrar(uint8_t patron) {
    PORTB = (PORTB & 0b11110000) | (patron & 0b00001111);
    PORTD = (PORTD & 0b10001111) | (patron & 0b01110000);
}
```

6.11. Eliminación de rebote por software

Los pulsadores mecánicos "rebotan" al presionarse: hacen contacto y lo rompen varias veces en pocos milisegundos antes de estabilizarse. Esto puede interpretarse como múltiples pulsaciones.

6.11.1. Método bloqueante (simple, para aprender)

Esperar un tiempo mayor que el rebote (∼20 ms) después de detectar el primer flanco:

```
if (!(PIND & (1 << PD2))) {          // detectar flanco de bajada
    _delay_ms(20);                    // esperar que termine el rebote
    if (!(PIND & (1 << PD2))) {       // verificar que sigue presionado
        // acción al presionar
    }
}
```

6.11.2. Método por detección de flanco (no bloqueante, TP5)

Se guarda el estado anterior del pin y se actúa solo cuando hay un **cambio de estado** (de 1 a 0 = flanco de bajada, de 0 a 1 = flanco de subida). El conteo de rebotes se hace en la ISR del timer:

```
volatile uint8_t estado_anterior = 1;   // con pull-up, inicial = 1 (suelto)
volatile uint8_t contador_rebote = 0;

// ISR cada 1 ms -- muestreo del pulsador
ISR(TIMER0_COMPA_vect) {
    uint8_t estado_actual = (PIND >> PD2) & 1;

    if (estado_actual != estado_anterior) {
        contador_rebote++;
        if (contador_rebote >= 20) {     // 20 lecturas estables = 20 ms
```

```
11              estado_anterior = estado_actual;
12              contador_rebote = 0;
13              if (estado_actual == 0) {
14                  // flanco de bajada confirmado -> pulsador
    presionado
15              }
16          }
17      } else {
18          contador_rebote = 0;
19      }
20  }
```

6.12. Resumen de registros del ATmega328 usados en los TPs

Registro	Función
DDRB, DDRC, DDRD	Dirección de datos de cada puerto
PORTB, PORTC, PORTD	Escritura de salidas / pull-ups de entradas
PINB, PINC, PIND	Lectura del estado de los pines
TCCR0A, TCCR0B	Control del Timer 0 (modo y preescalador)
TCCR1A, TCCR1B	Control del Timer 1
TIMSK0	Habilitación de interrupciones del Timer 0
TIMSK1	Habilitación de interrupciones del Timer 1
TCNT0	Valor actual del contador del Timer 0
OCR0A, OCR0B	Registros de comparación del Timer 0
SREG	Registro de estado (flags de la CPU, incluye el bit I de interrupciones globales)

Cuadro 6.3: Registros del ATmega328 utilizados en los TPs

Nota

`sei()` pone en 1 el bit I del `SREG` (habilita interrupciones globales).
`cli()` lo pone en 0 (deshabilita interrupciones globales).